



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA

TRABAJO FIN DE GRADO

**Estudio, implementación y perfilado de
FlashAttention
en GPU mediante CUDA, Triton y
PyTorch**

ÁNGEL MORENO DOMÍNGUEZ

Mérida, agosto de 2026



UNIVERSIDAD DE EXTREMADURA

CENTRO UNIVERSITARIO DE MÉRIDA

GRADO EN INGENIERÍA INFORMÁTICA

TRABAJO FIN DE GRADO

Estudio, implementación y perfilado de
FlashAttention
en GPU mediante CUDA, Triton y
PyTorch

Autor: Ángel Moreno Domínguez
Fdo.:

Tutores: Mercedes Eugenia Paoletti Ávila
Juan Mario Haut Hurtado
Fdo.:

Agradecimientos

Quiero dar las gracias a mi familia, cuyo apoyo silencioso y paciencia han sido la base de cada paso que he dado, y a mis amigos, que me recordaron que existe vida más allá de la pantalla.

Estoy profundamente agradecido a Manuel Ujaldón y Mercedes. Desde el principio confiaron en mí y vieron potencial donde yo solo veía incertidumbre. Consideraron que sería una buena idea que participara en una edición de PUMPS+AI, una experiencia que amplió mi visión del campo y me dio una confianza que no estaba seguro de tener.

Aunque a veces el panorama general pueda parecer sombrío, veo luz en el futuro, y buena parte de esa luz procede de las personas que me han acompañado durante el camino.

Quiero agradecer a Francisco Chávez las muchas horas que dedicó a resolver los problemas de la cola de Slurm. También estoy agradecido a COMPUTAEX y a Javier por aceptarme durante un mes completo de prácticas y por hacerme sentir, desde el primer día, como un miembro valorado del equipo. A Miguel Ángel le debo un agradecimiento especial por su apoyo durante una situación personal difícil hace dos años; su amabilidad llegó en el momento en que más la necesitaba.

También quiero dar las gracias a Javier, cuyas clases de EC disfruté sinceramente y que, sin que yo fuera consciente de ello en aquel momento, acabaron orientando la dirección de este Trabajo Fin de Grado.

Estoy profundamente agradecido a Mario por aceptar la dirección de este trabajo, así como por la orientación y la confianza que ha depositado en mí.

Quizá no haya sido el alumno más brillante, pero este trabajo refleja muchas horas de estudio, esfuerzo y perseverancia.

Resumen

El mecanismo de atención constituye uno de los componentes fundamentales de las arquitecturas *Transformer*, pero presenta un coste computacional cuadrático respecto a la longitud de la secuencia. FlashAttention reorganiza este cálculo para evitar materializar completamente la matriz de atención en memoria global y aprovechar de forma más eficiente la jerarquía de memoria de la GPU.

En este Trabajo Fin de Grado se estudia la implementación y optimización de FlashAttention sobre GPUs NVIDIA mediante dos niveles de abstracción diferentes. Por una parte, se desarrolla desde cero un kernel CUDA específico para arquitecturas Ampere, utilizando de forma explícita transferencias asíncronas mediante `cp.async`, cargas matriciales con `ldmatrix`, Tensor Cores mediante instrucciones `mma.sync`, memoria compartida con técnicas de *swizzling* y acumulación en FP32. Por otra parte, se implementa una versión equivalente utilizando Triton, delegando al compilador una parte importante de las decisiones relacionadas con la distribución del trabajo y la gestión de memoria.

Ambas implementaciones se comparan con SDPA de PyTorch, incluyendo sus rutas FlashAttention y cuDNN. La evaluación principal se realiza sobre una NVIDIA A100-SXM4-40GB utilizando entradas BF16, dimensión de cabeza igual a 128, un único elemento por *batch*, una única cabeza y atención no causal. Se utiliza un test funcional con `torch.allclose` para comprobar la corrección numérica, junto con pruebas de rendimiento para longitudes de secuencia crecientes y perfiles detallados mediante NVIDIA Nsight Compute.

Los resultados muestran que la implementación CUDA elimina los accesos globales no coalescentes y los conflictos de banco, evita *register spilling* y presenta un consumo de registros y memoria compartida inferior al de varias de las rutas comparadas. Sin embargo, estas optimizaciones locales no se traducen directamente en el mejor rendimiento. El análisis revela que la granularidad del trabajo y la estrategia de paralelización tienen un impacto determinante: el kernel CUDA procesa un bloque menor de consultas por CTA, lo que reduce la reutilización de K y V y limita el paralelismo disponible para

determinadas longitudes de secuencia. Triton muestra una mayor reutilización de datos. En cuDNN, el menor número de sectores solicitado es consistente con una mayor reutilización efectiva, aunque su *mapping* interno no se reconstruye en este trabajo. El perfil de SDPA, por su parte, muestra una organización de ejecución distinta y un mayor número de bloques.

Para longitudes de secuencia grandes, la implementación CUDA alcanza aproximadamente 100 TFLOP/s, frente a unos 126 TFLOP/s de Triton y valores superiores para SDPA y cuDNN. El trabajo muestra así que la optimización de kernels GPU no depende únicamente de minimizar accesos a memoria o consumo de recursos, sino de encontrar un equilibrio entre reutilización de datos, paralelismo, ocupación, sincronización y utilización efectiva de las unidades de cómputo.

Palabras clave – FlashAttention, CUDA, Triton, GPU, Tensor Cores, Ampere, optimización, Nsight Compute

Abstract

The attention mechanism is one of the fundamental components of Transformer architectures, but its computational cost grows quadratically with sequence length. FlashAttention reorganizes this computation to avoid fully materializing the attention matrix in global memory and to make more efficient use of the GPU memory hierarchy.

This Bachelor’s Thesis studies the implementation and optimization of FlashAttention on NVIDIA GPUs using two different levels of abstraction. On the one hand, a CUDA kernel is developed from scratch for Ampere architectures, explicitly using asynchronous transfers through `cp.async`, matrix loads with `ldmatrix`, Tensor Cores through `mma.sync` instructions, shared memory with *swizzling*, and FP32 accumulation. On the other hand, an equivalent implementation is developed using Triton, delegating a significant part of the decisions related to work distribution and memory management to the compiler.

Both implementations are compared against PyTorch’s `scaled_dot_product_attention`, including its FlashAttention and cuDNN execution paths. The main evaluation is performed on an NVIDIA A100-SXM4-40GB using BF16 inputs, a head dimension of 128, batch size one, one attention head, and non-causal attention. Functional tests verify numerical correctness, while performance benchmarks cover increasing sequence lengths and NVIDIA Nsight Compute is used for detailed profiling.

The results show that the CUDA implementation eliminates uncoalesced global memory accesses and shared-memory bank conflicts, avoids register spilling, and achieves lower register and shared-memory usage than several of the compared paths. However, these local optimizations do not directly translate into the best overall performance. The analysis reveals that work granularity and parallelization strategy have a decisive impact: the CUDA kernel processes a smaller query tile per CTA, which reduces the reuse of K and V and limits the amount of available parallelism for some sequence lengths. Triton and cuDNN achieve greater data reuse, whereas the SDPA profile exhibits a different execution organization and a larger number of

launched blocks.

For large sequence lengths, the CUDA implementation reaches approximately 100 TFLOP/s, compared with around 126 TFLOP/s for Triton and higher values for SDPA and cuDNN. The results therefore show that GPU kernel optimization does not depend solely on minimizing memory accesses or resource usage, but on balancing data reuse, parallelism, occupancy, synchronization, and effective utilization of the computational units.

Keywords – FlashAttention, CUDA, Triton, GPU, Tensor Cores, Ampere, optimization, Nsight Compute

Índice general

Índice de figuras	IX
-------------------	----

Índice de tablas	XIII
------------------	------

1. Motivación y objetivos	1
1.1. Introducción	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Originalidad del TFG	4
1.5. Planificación	5
1.6. Organización del documento	6
2. Estado del arte y Trabajos relacionados	7
2.1. Tecnologías relacionadas: Hardware	8
2.1.1. NVIDIA Blackwell (SM100 y SM103)	8
2.1.2. AMD Instinct (CDNA 3 y CDNA 4)	10
2.1.3. Tenstorrent	12
2.1.4. Cell Broadband Engine	13
2.2. Conceptos básicos: GPU	15
2.2.1. Doble buffer	15
2.2.2. cp.async.cg.16	17
2.2.3. ldmatrix.x4 y ldmatrix.x4.trans	18
2.2.4. Swizzling en memoria compartida	19
2.2.5. mma.sync.aligned.m16n8k16	20
2.3. Herramientas	22
2.3.1. C++20	22
2.3.2. CUDA Toolkit 13.0	22
2.3.3. clangd	22
2.3.4. clang-format	23
2.3.5. CMake	23
2.3.6. uv	23

2.3.7.	pytest	23
2.3.8.	Nsight Compute	24
2.3.9.	PyTorch	24
2.3.10.	Triton	24
2.3.11.	Ruff	24
2.3.12.	GoogleTest	25
2.3.13.	CTest	25
2.4.	Trabajos relacionados	25
2.5.	Contribuciones de este TFG	27
3.	Metodología de desarrollo	29
3.1.	FlashAttention y FlashAttention-2	29
3.1.1.	Algoritmo CUDA	31
3.1.2.	FlashAttention: Triton	34
4.	Entorno de prueba	37
4.1.	Hardware	37
4.2.	Entorno software	38
4.3.	Metodología de medida	39
5.	Tests	41
5.1.	Test funcional	41
5.2.	Tests no funcionales	42
5.2.1.	Tests de tiempo	42
5.2.2.	Tests de profiling	43
5.3.	Organización de las pruebas	44
6.	Análisis de los resultados	45
6.1.	Kernels observados en SDPA	45
6.2.	Registros y ocupación	46
6.3.	Memoria compartida y granularidad de lanzamiento	47
6.4.	Conflictos de banco	48
6.5.	Coalescencia de los accesos globales	48
6.6.	Número total de sectores solicitados	49
6.7.	Actividad de Tensor Cores y jerarquía de memoria	50
6.8.	Mediciones temporales	51
6.9.	Escalabilidad y throughput sostenido	54
6.10.	Lecciones aprendidas y posibles mejoras	55

7. Conclusiones	57
7.1. Limitaciones	59
7.2. Trabajo futuro	59
Anexos	61
A. Contexto complementario: mecanismo de atención y arquitectura GPT	63
A.1. Qué quiere decir la fórmula de la atención	63
A.2. Por qué la atención es importante	65
A.3. Encaje dentro de una arquitectura GPT	65
B. Instalación, compilación y reproducción de los experimentos	67
B.1. Organización del proyecto	67
B.2. Requisitos	68
B.3. Preparación del entorno Python	68
B.4. Configuración y compilación con CMake	68
B.5. Ejecución de tests	69
B.6. Perfilado con Nsight Compute	69
B.7. Registro del entorno de ejecución	70
B.8. Flujo recomendado para reproducir el trabajo	70
Bibliografía	71

Índice de figuras

6.1. Residencia permitida por recursos frente a trabajo disponible para el kernel CUDA.	47
6.2. Interpretación conceptual de la reutilización de un tile de K, V sobre las filas de consultas procesadas por un CTA.	50
6.3. Tiempo medio de ejecución frente a longitud de secuencia, con eje vertical lineal.	53

Índice de cuadros

4.1. Características de la GPU utilizada durante los experimentos.	38
4.2. Entorno software utilizado durante los experimentos.	39
6.1. Uso de registros por hilo para $N = 8192$	46
6.2. Ocupación teórica y alcanzada para $N = 8192$	46
6.3. Memoria compartida y granularidad de lanzamiento para $N =$ 8192.	47
6.4. Conflictos de banco observados en los kernels principales. . . .	48
6.5. Sectores globales reales, ideales y excesivos.	49
6.6. Número total de sectores globales solicitados.	49
6.7. Actividad aproximada del pipeline Tensor y señales de memoria para $N = 8192$	51
6.8. Tiempo medio y desviación típica de CUDA y Triton (ms). . .	52
6.9. Tiempo medio y desviación típica de SDPA y cuDNN (ms). . .	52
6.10. Throughput aproximado en el régimen de secuencias grandes. .	54

Capítulo 1

Motivación y objetivos

1.1. Introducción

En 2019 OpenAI presentó GPT-2, un modelo de lenguaje basado en la arquitectura *Transformer* [2]. A diferencia de arquitecturas recurrentes, los *Transformers* permiten procesar secuencias mediante un mecanismo de atención que relaciona directamente los distintos elementos de la entrada. Esta arquitectura fue presentada en *Attention Is All You Need* [1] y se ha convertido en uno de los componentes fundamentales de los modelos de lenguaje modernos.

Desde entonces, el tamaño de los modelos, la cantidad de datos utilizados durante el entrenamiento y la longitud de las secuencias procesadas han aumentado considerablemente. Esta evolución ha estado acompañada por un incremento igualmente importante de los recursos computacionales necesarios tanto para entrenamiento como para inferencia.

Uno de los principales costes de los modelos basados en *Transformers* se encuentra precisamente en el mecanismo de atención. Para una secuencia de longitud N , la matriz de puntuaciones

$$QK^T$$

contiene N^2 elementos. En la implementación convencional de la atención, esta matriz debe materializarse y posteriormente ser procesada por la operación de *softmax*, generando una cantidad considerable de tráfico a memoria.

El coste computacional de la atención es igualmente cuadrático respecto a la longitud de secuencia. Como consecuencia, el incremento de las ventanas de contexto utilizadas por los modelos actuales convierte esta operación en un objetivo especialmente relevante para la optimización.

Las unidades de procesamiento gráfico (GPU) se han convertido en el principal tipo de acelerador utilizado para estas cargas de trabajo. Su elevado paralelismo y la incorporación de unidades especializadas para operaciones matriciales permiten alcanzar un gran rendimiento, pero obtener dicho rendimiento requiere considerar explícitamente aspectos como la jerarquía de memoria, la distribución del trabajo, la sincronización entre hilos y la utilización de los recursos internos de cada multiprocesador.

Paralelamente, han aparecido lenguajes y compiladores especializados para facilitar el desarrollo de kernels de alto rendimiento y automatizar parte de su optimización. Entre ellos se encuentra Triton, utilizado en este trabajo junto con CUDA y con implementaciones de referencia de PyTorch.

1.2. Motivación

FlashAttention constituye un ejemplo especialmente adecuado para estudiar este problema. El algoritmo reorganiza el cálculo de la atención para evitar la materialización completa de las matrices intermedias en memoria global. Para ello, divide las matrices Q , K y V en bloques y mantiene los resultados intermedios en niveles más rápidos de la jerarquía de memoria [16].

Esta estrategia reduce considerablemente el tráfico hacia memoria global, pero traslada parte de la complejidad al diseño del kernel. Una implementación eficiente debe decidir cómo distribuir los bloques entre los *warps*, qué datos conservar en registros o memoria compartida, cómo realizar las transferencias entre niveles de memoria y cómo alimentar de forma continua las unidades matriciales de la GPU.

La arquitectura Ampere proporciona mecanismos específicamente diseñados para este tipo de cargas de trabajo [5]. Entre ellos se encuentran las instrucciones asíncronas `cp.async`, las cargas colectivas `ldmatrix` y las instrucciones `mma.sync` empleadas para ejecutar productos matriciales sobre los Tensor Cores; su semántica se documenta en la ISA PTX de NVIDIA [7].

El control directo de estas instrucciones mediante CUDA permite estudiar detalladamente su comportamiento, pero también incrementa considerablemente la complejidad de implementación. Triton, por el contrario, permite expresar gran parte del cálculo utilizando operaciones matriciales de alto nivel y delegar al compilador decisiones como el reparto de los datos, el uso de memoria compartida o la generación de determinadas instrucciones de bajo nivel.

La motivación principal de este trabajo consiste, por tanto, en implementar FlashAttention utilizando ambos enfoques y estudiar las diferencias entre ellos. El interés no reside únicamente en determinar qué versión obtiene

un menor tiempo de ejecución, sino en relacionar el rendimiento observado con decisiones concretas de organización del trabajo, movimiento de datos y utilización de los recursos de la GPU.

La comparación se completa utilizando como referencia `scaled_dot_product_attention` (SDPA) de PyTorch [13] y su backend basado en cuDNN [14]. Estas rutas permiten situar el rendimiento obtenido por los kernels desarrollados dentro de un contexto realista.

1.3. Objetivos

El objetivo general del trabajo consiste en estudiar la implementación y optimización de FlashAttention sobre GPUs NVIDIA, comparando una implementación propia en CUDA, una implementación desarrollada con Triton y las rutas de referencia disponibles mediante PyTorch SDPA y cuDNN.

A partir de este objetivo general se establecen los siguientes objetivos específicos:

- Estudiar el funcionamiento del mecanismo de atención utilizado por los modelos *Transformer* y analizar los principales costes computacionales y de memoria asociados a su implementación convencional.
- Estudiar el algoritmo FlashAttention y, en particular, la utilización del *softmax online* para evitar la materialización completa de la matriz de atención.
- Desarrollar una implementación de FlashAttention directamente en CUDA para GPUs NVIDIA de arquitectura Ampere y dimensión de cabeza $d = 128$.
- Utilizar explícitamente los Tensor Cores mediante instrucciones MMA con operandos BF16 y acumulación FP32.
- Implementar un *pipeline* de transferencia entre memoria global y memoria compartida mediante instrucciones `cp.async` y múltiples buffers.
- Utilizar instrucciones `ldmatrix` para transferir los fragmentos necesarios desde memoria compartida al fichero de registros.
- Diseñar un patrón de *swizzling* para la memoria compartida que permita evitar conflictos de banco durante las cargas matriciales.

- Optimizar los accesos a memoria global de Q , K , V y O , buscando que las transferencias realizadas por cada *warp* sean completamente coalescentes.
- Desarrollar una segunda implementación equivalente utilizando Triton y explorar sus parámetros de ejecución.
- Utilizar el mecanismo de *autotuning* de Triton para estudiar diferentes configuraciones de tamaño de bloque, número de *warps* y etapas del *pipeline*.
- Implementar un test funcional de referencia que compruebe la correctitud numérica de los kernels mediante `torch.allclose` frente a SDPA.
- Desarrollar tests no funcionales destinados a medir el tiempo de ejecución y estudiar la escalabilidad de las distintas implementaciones al incrementar la longitud de secuencia.
- Analizar los kernels utilizando NVIDIA Nsight Compute, prestando especial atención a métricas relacionadas con ocupación, utilización de registros y memoria compartida, accesos no coalescentes, conflictos de banco, utilización de los Tensor Cores y causas de espera de los *warps*.
- Comparar el rendimiento de las implementaciones CUDA y Triton con SDPA y con el backend basado en cuDNN.
- Identificar las decisiones de diseño que limitan el rendimiento de la implementación CUDA y proponer posibles líneas de mejora.

1.4. Originalidad del TFG

La principal aportación del trabajo reside en el desarrollo desde cero de una implementación de FlashAttention directamente sobre CUDA utilizando primitivas específicas de la arquitectura Ampere.

En lugar de limitar la comparación a tiempos de ejecución de implementaciones existentes, se estudia el algoritmo desde el nivel de las instrucciones ejecutadas por la GPU. Esto incluye el reparto explícito del trabajo entre *warps*, la utilización del fichero de registros y de la memoria compartida, la construcción de un *pipeline* mediante transferencias asíncronas, el acceso a Tensor Cores y la reorganización física de los datos mediante *swizzling*.

La implementación CUDA se compara con una versión funcionalmente equivalente desarrollada mediante Triton y con las implementaciones de

referencia de PyTorch. La aportación no se limita a ordenar estas rutas por tiempo de ejecución, sino que busca relacionar sus diferencias con métricas hardware obtenidas mediante perfilado.

Finalmente, el análisis mediante Nsight Compute permite relacionar las diferencias de rendimiento con características concretas de los kernels, en lugar de limitar la evaluación a una comparación puramente temporal.

1.5. Planificación

El desarrollo del proyecto se ha realizado de forma incremental. En una primera fase se estudió el algoritmo de atención y el funcionamiento de FlashAttention, prestando especial atención al *softmax online* y a la descomposición por bloques necesaria para evitar la materialización de la matriz completa de atención.

Posteriormente se desarrolló una primera implementación CUDA funcional. Sobre esta versión se fueron incorporando progresivamente las distintas optimizaciones de bajo nivel.

Entre las principales etapas del desarrollo se encuentran:

1. implementación del producto QK^T utilizando Tensor Cores;
2. implementación del *softmax online*;
3. acumulación del producto PV manteniendo la salida en FP32;
4. introducción de transferencias asíncronas mediante `cp.async`;
5. utilización de múltiples buffers en memoria compartida;
6. incorporación de las cargas mediante `ldmatrix`;
7. diseño del patrón de *swizzling* de memoria compartida;
8. reorganización de las cargas globales de Q , K y V para eliminar accesos no coalescentes;
9. reorganización de la escritura de O para conseguir stores coalescentes;
10. análisis del consumo de registros, memoria compartida y ocupación;
11. desarrollo de la implementación Triton y exploración de sus parámetros de ejecución;
12. desarrollo del test funcional y de los tests de rendimiento;

13. evaluación de las implementaciones mediante NVIDIA Nsight Compute;
14. comparación final con SDPA y cuDNN.

El proceso de optimización ha sido iterativo. Después de cada modificación se verificaba en primer lugar la correctitud del kernel mediante el test funcional y, posteriormente, se analizaba su efecto sobre el rendimiento y las métricas obtenidas mediante el *profiler*. Esta metodología permitió evitar que una optimización de rendimiento introdujera errores silenciosos en el resultado.

1.6. Organización del documento

El resto de la memoria se organiza de la siguiente forma.

En primer lugar, se presentan los conceptos necesarios para comprender el mecanismo de atención, la arquitectura *Transformer*, FlashAttention y las características de las GPUs NVIDIA relevantes para su implementación.

Posteriormente se describe la metodología de desarrollo utilizada. Se presenta el algoritmo FlashAttention empleado como referencia y se detallan las implementaciones CUDA y Triton, incluyendo la distribución del trabajo, la utilización de la jerarquía de memoria, los Tensor Cores y las principales optimizaciones aplicadas.

A continuación se describe el entorno de prueba utilizado para realizar las mediciones, incluyendo las características de la GPU NVIDIA A100, el entorno software y las versiones de las herramientas empleadas.

El siguiente capítulo presenta el test funcional utilizado para verificar la correctitud y los tests no funcionales empleados en las medidas temporales y en el *profiling*.

Posteriormente se realiza el análisis de los resultados obtenidos. Se comparan las implementaciones CUDA, Triton, SDPA y cuDNN mediante sus tiempos de ejecución y mediante las métricas obtenidas con NVIDIA Nsight Compute. Se estudian aspectos como la ocupación, el número de registros, el consumo de memoria compartida, los conflictos de banco, los accesos coalescentes y el número total de sectores de memoria solicitados.

Finalmente, se presentan las conclusiones del trabajo, las principales lecciones obtenidas durante el desarrollo y las posibles líneas de investigación y optimización futuras.

Capítulo 2

Estado del arte y Trabajos relacionados

La inteligencia artificial ha experimentado en los últimos años un crecimiento extraordinario tanto en inversión como en investigación. Con una frecuencia cada vez mayor, compañías como OpenAI y Anthropic, así como organizaciones y proyectos centrados en modelos abiertos, como DeepSeek o Kimi, presentan nuevos modelos que incorporan arquitecturas, técnicas de entrenamiento y mecanismos de inferencia diferentes.

Esta evolución no se limita únicamente al diseño de los modelos. El hardware empleado para su entrenamiento y ejecución también se encuentra en constante desarrollo, con el objetivo de mejorar tanto el rendimiento como la eficiencia energética. En este contexto coexisten soluciones muy diversas, como las GPU desarrolladas por NVIDIA y AMD, las TPU de Google, los aceleradores de Tenstorrent, los procesadores de Apple, los chips MTIA de Meta, los sistemas de Cerebras y otros aceleradores de propósito específico, además de extensiones incorporadas en procesadores de propósito general, como las instrucciones AMX de Intel. Cada una de estas propuestas presenta una arquitectura diferente, con ventajas, limitaciones y compromisos propios.

De forma paralela, surgen nuevas herramientas, lenguajes y compiladores orientados a aprovechar de manera más eficiente las capacidades específicas de cada arquitectura hardware, al mismo tiempo que intentan reducir la complejidad de su programación. La rápida evolución conjunta de los modelos, el hardware y las herramientas de desarrollo hace especialmente difícil abarcar de forma exhaustiva el estado del arte. Por este motivo, este trabajo se centra en un conjunto concreto de tecnologías y técnicas representativas, en lugar de pretender ofrecer una revisión completa de un campo que se encuentra en continua y rápida evolución.

En este capítulo se presentan algunas de estas tecnologías, describiendo

brevemente sus principales características, ventajas y limitaciones. Por un lado, se analizan distintas soluciones hardware orientadas a la aceleración de cargas de inteligencia artificial y, por otro, herramientas, lenguajes y compiladores diseñados para facilitar su programación y aprovechar de forma eficiente las capacidades del hardware disponible.

2.1. Tecnologías relacionadas: Hardware

2.1.1. NVIDIA Blackwell (SM100 y SM103)

Las GPU NVIDIA Blackwell orientadas a centros de datos constituyen una generación posterior a Ampere y amplían de forma importante los mecanismos disponibles para organizar el movimiento de datos y el cómputo. Las características descritas en esta sección se apoyan en la documentación técnica y las guías de programación de NVIDIA para Blackwell [8, 7].

Con respecto a Ampere, arquitectura sobre la que se desarrolla la implementación CUDA de este trabajo, Blackwell incorpora y amplía diferentes mecanismos destinados a mejorar el movimiento de datos, la utilización de los Tensor Cores y la capacidad de solapar distintas etapas de un kernel. Entre ellos destacan los siguientes:

- **Warp specialization.** La especialización de warps consiste en asignar funciones diferentes a distintos warps de un mismo bloque. Aunque la técnica no es exclusiva de Blackwell, las nuevas operaciones asíncronas de las generaciones más recientes permiten explotarla de forma especialmente efectiva. Por ejemplo, algunos warps pueden actuar como productores encargados de iniciar transferencias de memoria mediante TMA, mientras que otros actúan como consumidores ejecutando operaciones sobre los Tensor Cores mediante instrucciones `tcgen05.mma`. Otros warps pueden encargarse de etapas adicionales del algoritmo, como el cálculo del softmax o del epílogo. De esta forma, las distintas fases del kernel pueden ejecutarse de manera solapada y formar un pipeline productor-consumidor.
- **Tensor Memory (TMEM).** En Ampere, los acumuladores producidos por las instrucciones MMA se almacenan directamente en registros. Blackwell introduce Tensor Memory, una memoria dedicada a las operaciones de los Tensor Cores. En SM100, esta memoria se organiza como una matriz de 128 filas y 512 columnas por CTA, donde cada posición contiene 32 bits, lo que supone un total de 256 KiB. Los resultados de

las operaciones `tcgen05.mma` pueden almacenarse y acumularse directamente en TMEM, reduciendo la presión sobre el fichero de registros. Posteriormente, estos datos pueden transferirse a registros cuando sea necesario realizar operaciones adicionales, como las correspondientes al epílogo.

- **Tensor Memory Accelerator (TMA).** TMA es un mecanismo hardware para realizar transferencias asíncronas de datos entre memoria global y memoria compartida. A diferencia de las operaciones utilizadas en Ampere, el programador puede describir la disposición de un tensor mediante un *tensor map*, especificando sus dimensiones, strides y otras propiedades. El hardware se encarga entonces de calcular las direcciones necesarias para realizar la transferencia. TMA también permite aplicar directamente patrones de *swizzling* durante la copia a memoria compartida, evitando que el programador tenga que calcular manualmente estas transformaciones y facilitando la reducción de conflictos entre bancos de memoria.
- **Thread Block Clusters y Distributed Shared Memory.** Los *Thread Block Clusters*, introducidos originalmente en Hopper y soportados también por Blackwell, permiten agrupar varios bloques de hilos dentro de un mismo cluster. Los bloques pertenecientes a este cluster pueden acceder a la memoria compartida de otros bloques mediante un espacio denominado *Distributed Shared Memory* (DSM). Esto permite realizar lecturas, escrituras y operaciones atómicas sobre la memoria compartida de bloques que pueden estar ejecutándose en distintos Streaming Multiprocessors, evitando en determinados algoritmos la necesidad de utilizar memoria global como mecanismo intermedio de comunicación. Aunque este acceso resulta considerablemente más eficiente que intercambiar los datos a través de memoria global, acceder a la memoria compartida de otro SM presenta una latencia superior a la de la memoria compartida local.
- **MBarrier.** Las *mbarriers* constituyen un mecanismo de sincronización especialmente útil para implementar pipelines productor-consumidor junto con operaciones asíncronas como TMA. Una barrera se inicializa indicando el número esperado de participantes. Los hilos notifican su llegada a la barrera y, en el caso de operaciones asíncronas, también puede asociarse a ella una cantidad de datos pendientes de transferir. La fase de la barrera se considera completada únicamente cuando se han producido las llegadas esperadas y han finalizado todas las transacciones asociadas.

Este mecanismo permite implementar de forma eficiente esquemas de *multibuffering*. Por ejemplo, cada buffer puede disponer de una barrera que indique que contiene datos listos para ser consumidos y otra que indique que puede volver a ser utilizado por el productor. Antes de leer un buffer, el consumidor espera a que finalice su barrera de datos disponibles; una vez procesados los datos, señala que el buffer puede reutilizarse. De forma análoga, el productor espera a que el buffer esté libre antes de iniciar una nueva transferencia. De este modo, movimiento de datos y cómputo pueden solaparse sin necesidad de sincronizar globalmente todos los warps.

- **Tensor Cores de quinta generación (TCGen05).** Blackwell introduce una nueva generación de Tensor Cores accesible mediante las instrucciones `tcgen05`. A diferencia de las instrucciones MMA utilizadas en Ampere, estas operaciones están diseñadas para trabajar conjuntamente con Tensor Memory y memoria compartida. Los acumuladores se almacenan directamente en Tensor Memory, mientras que los operandos pueden obtenerse de memoria compartida y, en determinadas variantes, también de Tensor Memory. Esto permite realizar operaciones matriciales de mayores dimensiones sin mantener los acumuladores en el fichero general de registros, reduciendo así la presión sobre estos. Además, incorporan soporte hardware para nuevos formatos numéricos y operaciones de *block scaling*, permitiendo realizar productos matriciales de muy baja precisión de forma eficiente.

Estas características modifican considerablemente la forma en la que pueden estructurarse kernels de alto rendimiento con respecto a Ampere. En particular, permiten separar de forma más clara el movimiento de datos y el cómputo, reducir la presión sobre los registros y construir pipelines asíncronos en los que diferentes unidades hardware trabajan simultáneamente.

2.1.2. AMD Instinct (CDNA 3 y CDNA 4)

AMD desarrolla la familia de aceleradores Instinct, orientada específicamente a cargas de inteligencia artificial y computación de alto rendimiento (HPC). Las generaciones recientes se basan en las arquitecturas CDNA 3 y CDNA 4, cuya organización y características se describen en la documentación oficial de AMD [9]. Aunque su modelo de programación presenta ciertas similitudes con el de las GPU de NVIDIA, existen diferencias relevantes tanto en la organización del hardware como en las instrucciones disponibles.

AMD no constituye el foco principal de este trabajo, por lo que su arquitectura no se analiza en profundidad. No obstante, se incluye dentro del

estado del arte debido a su relevancia actual en el ámbito de los aceleradores para inteligencia artificial y como posible línea de trabajo futura. Por este motivo, las descripciones que se presentan a continuación tienen un carácter introductorio y superficial.

- **Wavefronts y Compute Units.** Mientras que las arquitecturas NVIDIA agrupan los hilos en warps de 32 hilos que se ejecutan sobre un Streaming Multiprocessor, las arquitecturas CDNA emplean *wavefronts* de 64 hilos y organizan el hardware en *Compute Units* (CU). Todos los hilos de un wavefront ejecutan una misma secuencia de instrucciones, de forma análoga al modelo SIMT utilizado por NVIDIA.
- **LDS y registros.** El equivalente aproximado de la memoria compartida de CUDA es el *Local Data Share* (LDS), una memoria gestionada explícitamente por software y compartida por los wavefronts pertenecientes a un mismo workgroup. AMD distingue además entre registros escalares (SGPR), utilizados para valores comunes al wavefront, y registros vectoriales (VGPR), que almacenan valores diferentes para cada hilo. Esta separación refleja de forma explícita la distinción entre operaciones uniformes y vectoriales de la arquitectura.
- **Matrix Cores y MFMA.** Las arquitecturas CDNA incorporan unidades especializadas para operaciones matriciales, denominadas Matrix Cores. Estas unidades se utilizan mediante instrucciones *Matrix Fused Multiply-Add* (MFMA), que realizan productos y acumulaciones matriciales de forma cooperativa entre todos los hilos de un wavefront. Estas instrucciones desempeñan un papel similar a las instrucciones MMA de las GPU NVIDIA. En CDNA 4 se amplía considerablemente su rendimiento y se incorpora soporte nativo para formatos numéricos de baja precisión utilizados en inteligencia artificial.
- **Formatos de baja precisión.** CDNA 4 incorpora soporte para formatos como FP8, MXFP8, MXFP6 y MXFP4, además de operaciones con *sparsity*. El uso de estas representaciones permite reducir el volumen de datos transferido y aumentar el rendimiento de los Matrix Cores, resultando especialmente interesante para el entrenamiento y la inferencia de modelos de gran tamaño.
- **Arquitectura basada en chiplets.** Los aceleradores Instinct modernos están contruidos utilizando múltiples *Accelerator Complex Dies* (XCD) interconectados mediante AMD Infinity Fabric. Por ejemplo, la MI300X integra hasta ocho XCD junto con memoria HBM3. Esta organización

permite construir aceleradores de gran tamaño sin necesidad de fabricar toda la GPU como un único dado monolítico.

Este enfoque basado en *chipllets*, utilizado también por AMD en sus procesadores Ryzen, permite ensamblar dispositivos de gran tamaño a partir de varios dados de menor superficie. Esto puede mejorar el rendimiento de fabricación, ya que un defecto localizado afecta únicamente a uno de los *chipllets* y no necesariamente al dispositivo completo. Como contrapartida, la comunicación entre *chipllets* introduce una latencia y un coste energético superiores a los de la comunicación dentro de un mismo dado, por lo que el diseño de la interconexión resulta especialmente importante para mantener un elevado rendimiento.

- **Memoria HBM.** Una de las principales características de los aceleradores Instinct es su elevada capacidad y ancho de banda de memoria. La MI300X dispone de 192 GB de HBM3 con un ancho de banda máximo de 5,3 TB/s, mientras que la serie MI350 aumenta estas cifras hasta 288 GB de HBM3E y 8 TB/s. Esta elevada capacidad resulta especialmente relevante en modelos de lenguaje de gran tamaño, donde el almacenamiento de los pesos y la reducción del tráfico entre dispositivos pueden condicionar considerablemente el rendimiento.

AMD ha destacado especialmente por ofrecer una gran capacidad de memoria HBM por acelerador. Como referencia, una NVIDIA B200 dispone de 180 GB de HBM3E, frente a los 192 GB de la MI300X. Sin embargo, con Blackwell Ultra esta diferencia se reduce, ya que la B300 alcanza también los 288 GB de HBM3E de la MI350X. Por tanto, aunque AMD ha mantenido tradicionalmente una ventaja en capacidad de memoria en algunas generaciones, las propuestas más recientes de ambos fabricantes presentan cifras muy similares en este aspecto.

- **ROCm y HIP.** El ecosistema software equivalente a CUDA en AMD se denomina ROCm. Dentro de este, HIP proporciona un modelo de programación C++ similar a CUDA que permite desarrollar kernels para GPU AMD. ROCm incluye además compiladores basados en LLVM, bibliotecas optimizadas para operaciones matriciales y primitivas utilizadas habitualmente en aprendizaje automático.

2.1.3. Tenstorrent

Tenstorrent desarrolla aceleradores específicos para inteligencia artificial basados en núcleos Tensix, que combinan unidades de cómputo matricial con procesadores RISC-V programables. Su modelo de programación está

orientado a controlar de forma explícita tanto el movimiento de datos como el cómputo. [10]

Las tarjetas Blackhole integran 120 núcleos Tensix, alcanzan hasta 664 TFLOPS en formatos de baja precisión y disponen de 180 MB de SRAM distribuida junto a las unidades de cómputo. Esta cantidad de SRAM es elevada en comparación con una GPU convencional, aunque su memoria externa es mucho más limitada: hasta 32 GB de GDDR6 y unos 512 GB/s.

Tenstorrent también destaca por ofrecer tarjetas con un precio de entrada considerablemente inferior al de los aceleradores de datacenter de NVIDIA y AMD. A cambio, sus prestaciones, capacidad de memoria externa y ecosistema software son también más reducidos.

- **Tensix Cores.** Los aceleradores Blackhole actuales integran 120 núcleos Tensix. Cada uno de ellos combina unidades de cómputo matricial, procesadores RISC-V y memoria SRAM local, siguiendo una organización diferente a la de las GPU tradicionales.
- **Capacidad de cómputo.** Las tarjetas Blackhole alcanzan hasta 664 TFLOPS utilizando el formato *Block FP8*, estando orientadas principalmente a operaciones matriciales de baja precisión utilizadas en modelos de inteligencia artificial.
- **SRAM.** Cada chip Blackhole dispone de 180 MB de SRAM, aproximadamente 1,5 MB por núcleo Tensix. Esta memoria actúa como un *scratchpad* gestionado explícitamente y permite mantener datos próximos a las unidades de cómputo, reduciendo el tráfico hacia memoria externa.
- **Memoria externa e interconexión.** Las tarjetas Blackhole incorporan hasta 32 GB de memoria GDDR6 y alcanzan aproximadamente 512 GB/s de ancho de banda. Determinados modelos incluyen además enlaces dedicados para conectar varios aceleradores entre sí, permitiendo construir sistemas con un mayor número de núcleos y memoria agregada.

2.1.4. Cell Broadband Engine

Como contexto histórico, resulta interesante mencionar el *Cell Broadband Engine*, desarrollado conjuntamente por IBM, Sony y Toshiba a mediados de la década de 2000. Aunque se trata de una arquitectura con más de veinte años de antigüedad, presenta numerosas similitudes con técnicas utilizadas actualmente en GPU y aceleradores especializados. [11]

El procesador combinaba un núcleo de propósito general, denominado *Power Processing Element* (PPE), con ocho *Synergistic Processing Elements* (SPE) orientados al cálculo vectorial. Algunas de sus características más destacables son:

- **Memoria SRAM local.** Cada SPE disponía de 256 KB de memoria SRAM local, denominada *Local Store*. Tanto las instrucciones como los datos utilizados por el SPE debían encontrarse en esta memoria, que era gestionada explícitamente por software. Este diseño presenta similitudes con el uso de memorias SRAM gestionadas explícitamente en las GPU actuales, como la memoria compartida de los Streaming Multiprocessors. La principal diferencia frente a una caché convencional es que el programador decide explícitamente qué datos se encuentran almacenados en ella.
- **Transferencias entre SPE.** Los motores DMA no estaban limitados a realizar transferencias entre memoria principal y la memoria local de un SPE. También era posible transferir datos directamente entre los *Local Stores* de diferentes SPE. Esta posibilidad permitía comunicar directamente distintas unidades de cómputo sin utilizar la memoria principal como almacenamiento intermedio, una idea que recuerda a mecanismos modernos de comunicación entre unidades de procesamiento, como la *Distributed Shared Memory* disponible en arquitecturas NVIDIA recientes.
- **DMA asíncrono y double buffering.** Cada SPE disponía de un *Memory Flow Controller* capaz de ejecutar transferencias DMA de manera asíncrona y en paralelo con el procesador. Esto permitía utilizar técnicas de *double buffering*: mientras el SPE procesaba los datos almacenados en un buffer, el motor DMA podía cargar simultáneamente los datos necesarios para la siguiente iteración en un segundo buffer. Al terminar el cálculo, los buffers intercambiaban sus funciones. Este mecanismo permitía ocultar parcialmente la latencia de memoria mediante el solapamiento entre transferencia y cómputo.
- **Eventos asociados a las transferencias.** Las operaciones DMA podían agruparse mediante identificadores o *tags*, permitiendo comprobar o esperar de forma conjunta a su finalización. El *Memory Flow Controller* podía además generar eventos cuando se actualizaba el estado de estos grupos de transferencias, eventos que podían ser tratados por el SPE mediante su mecanismo de eventos e interrupciones. De esta

forma, no era necesario estructurar todo el programa como una espera activa continua sobre las transferencias de memoria.

Resulta especialmente llamativo que un procesador diseñado alrededor de 2005 incorporase mecanismos tan similares a los empleados en aceleradores actuales. El uso de SRAM gestionada explícitamente, transferencias asíncronas, comunicación directa entre unidades de cómputo y técnicas de *double buffering* aparecen de nuevo, con implementaciones más sofisticadas, en arquitecturas modernas como NVIDIA Hopper y Blackwell. El Cell Broadband Engine constituye, por tanto, un ejemplo histórico de cómo muchas de las técnicas utilizadas actualmente para maximizar el rendimiento no son conceptos nuevos, sino evoluciones de ideas presentes desde hace décadas.

2.2. Conceptos básicos: GPU

Los conceptos de modelo de ejecución, jerarquía de memoria, sincronización y memoria compartida utilizados en esta sección siguen la documentación oficial de CUDA [6].

En esta sección se describen algunos de los conceptos e instrucciones de la arquitectura NVIDIA Ampere que resultan fundamentales para comprender la implementación CUDA desarrollada en este trabajo. Se incluyen únicamente los mecanismos que intervienen directamente en el kernel y en la interpretación posterior de los perfiles.

La sección no pretende constituir un manual completo de programación de GPU. Se presuponen conocimientos básicos de arquitectura de computadores, programación paralela y funcionamiento general de las GPU. El objetivo es únicamente introducir aquellos conceptos necesarios para facilitar la lectura y comprensión del código CUDA presentado posteriormente.

2.2.1. Doble buffer

El *double buffering* es una técnica utilizada para solapar la producción y el consumo de datos. La idea consiste en disponer de dos buffers que se utilizan de forma alterna:

1. El productor comienza a rellenar los buffers.
2. Cuando uno de ellos contiene los datos necesarios, el consumidor comienza a procesarlo mientras el productor continúa preparando el siguiente buffer.

3. Una vez finalizado el consumo del primer buffer, los roles se intercambian y el proceso se repite.

El objetivo es evitar que el consumidor tenga que esperar a que finalice una transferencia de memoria antes de comenzar el siguiente cálculo. Si el productor es suficientemente rápido como para preparar el siguiente buffer antes de que el consumidor termine de procesar el actual, el coste de transferencia puede quedar parcialmente o incluso completamente oculto por el cómputo.

Por el contrario, si la producción de datos es más lenta que su consumo, el consumidor acabará alcanzando al productor y tendrá que esperar a que el siguiente buffer esté disponible. En el caso opuesto, si el consumidor es considerablemente más lento, será el productor quien termine esperando a que quede libre un buffer antes de poder reutilizarlo. En ambos casos disminuye el grado de solapamiento entre producción y consumo.

Multibuffering

El *multibuffering* generaliza la técnica de doble buffer utilizando más de dos buffers en rotación. Su objetivo es aumentar la distancia entre el productor y el consumidor, de forma que el consumidor disponga de varios bloques de datos preparados antes de necesitarlos.

Esta técnica resulta especialmente útil en GPU, donde numerosos warps pueden iniciar transferencias de memoria de forma simultánea y competir por el ancho de banda disponible. En un escenario desfavorable, si cada warp solicita bloques de gran tamaño, una transferencia individual puede ocupar durante más tiempo los recursos del subsistema de memoria, incrementando la latencia experimentada por el resto de solicitudes.

Reducir el tamaño de los buffers permite dividir el tráfico en transferencias de menor granularidad, favoreciendo que un mayor número de solicitudes procedentes de distintos warps puedan avanzar de forma intercalada a través de la tubería de memoria. Si esta reducción se combina con un mayor número de buffers, es posible mantener varias transferencias en vuelo y proporcionar un mayor margen para ocultar las variaciones de latencia antes de que el consumidor alcance al productor.

Sin embargo, aumentar el número de buffers o reducir excesivamente su tamaño también introduce costes. Un mayor número de buffers incrementa el consumo de memoria local, mientras que buffers más pequeños requieren más transferencias y más puntos de sincronización. Por tanto, la elección del número y tamaño de los buffers constituye un compromiso entre latencia, ancho de banda, utilización de memoria y coste de sincronización.

Debido a la dificultad de obtener una solución óptima de forma analítica, en la práctica suele evaluarse experimentalmente un conjunto de configuraciones y seleccionar aquella que ofrece el mejor rendimiento para el dispositivo y la carga de trabajo considerados.

2.2.2. `cp.async.cg.16`

La instrucción `cp.async` permite iniciar una copia asíncrona desde memoria global hacia memoria compartida [7]. El hilo que emite la instrucción puede continuar ejecutando otras instrucciones antes de que la transferencia haya finalizado, siendo necesario sincronizar posteriormente antes de consumir los datos copiados.

Una de sus principales ventajas frente a una secuencia convencional de carga y almacenamiento es que permite trasladar los datos directamente desde memoria global a memoria compartida, evitando utilizar el fichero de registros como almacenamiento intermedio. La variante `cp.async.cg.16` utilizada en este trabajo copia 16 bytes por instrucción y evita la caché L1, manteniendo los accesos cacheables en L2.

Aunque cada instrucción copie únicamente 16 bytes, sigue siendo importante que los accesos realizados por los hilos de un warp sean contiguos y estén correctamente alineados. La caché L2 se organiza en líneas de 128 bytes, subdivididas en sectores de 32 bytes. Cuando los accesos de los hilos son contiguos, varias solicitudes pueden ser atendidas utilizando pocos sectores pertenecientes a las mismas líneas de caché. Por el contrario, un patrón disperso puede obligar a acceder a un número mucho mayor de sectores y líneas, incrementando el tráfico de memoria.

En el peor caso, si numerosos hilos acceden simultáneamente a posiciones no contiguas, pueden introducirse en L2 muchas líneas distintas para satisfacer una pequeña cantidad de datos útiles de cada una. Esta presión adicional sobre la caché puede provocar la expulsión de líneas que todavía podrían reutilizarse. Como consecuencia, un acceso posterior a los siguientes 16 bytes contiguos puede encontrarse con que la información correspondiente ya no reside en L2 y debe recuperarse nuevamente desde niveles inferiores de la jerarquía de memoria.

Por este motivo, el uso de `cp.async` no elimina la necesidad de realizar accesos coalescentes. La instrucción optimiza el camino entre memoria global y memoria compartida y facilita el solapamiento entre transferencia y cómputo, pero el patrón de acceso de los hilos continúa siendo determinante para utilizar eficientemente el ancho de banda disponible.

En el kernel cuda, `cp.async` se utiliza para el multibuffer de K/V.

2.2.3. `ldmatrix.x4` y `ldmatrix.x4.trans`

La instrucción `ldmatrix` permite cargar de forma cooperativa matrices desde memoria compartida directamente a los registros de un warp [7]. Está diseñada principalmente para preparar los operandos que posteriormente serán utilizados por las instrucciones MMA de los Tensor Cores.

En este trabajo se utilizan matrices de 8×8 elementos de 16 bits. Como cada registro tiene un tamaño de 32 bits, cada registro recibido por un thread contiene dos elementos BF16 consecutivos. La carga se realiza cooperativamente entre los 32 threads del warp, de forma que cada grupo de threads proporciona las direcciones necesarias para recuperar las diferentes filas de las matrices.

El modificador `x4` permite cargar cuatro matrices 8×8 mediante una única instrucción. En el kernel desarrollado, las direcciones se preparan de manera que estas cuatro matrices corresponden a los cuatro subtiles de un tile lógico de 16×16 . El orden de carga puede visualizarse como un recorrido en forma de “Z”: primero se recorren las dos mitades verticales correspondientes a las primeras ocho columnas y, posteriormente, las correspondientes a las ocho columnas siguientes:

```
matrix 0 -> rows 0..7, columns 0..7
matrix 1 -> rows 8..15, columns 0..7
matrix 2 -> rows 0..7, columns 8..15
matrix 3 -> rows 8..15, columns 8..15
```

Es decir, primero se carga la parte superior izquierda del tile, después la parte inferior izquierda, a continuación la parte superior derecha y finalmente la parte inferior derecha. Esta disposición resulta especialmente importante, ya que los cuatro registros devueltos por `ldmatrix.x4` no representan directamente dos matrices 16×8 , sino cuatro fragmentos 8×8 que deben reagruparse posteriormente de acuerdo con el operando requerido por la instrucción MMA.

Los threads 0–7 proporcionan las direcciones de las ocho filas de la primera matriz, los threads 8–15 las correspondientes a la segunda, los threads 16–23 las de la tercera y los threads 24–31 las de la cuarta. Como resultado, cada thread recibe cuatro registros de 32 bits, uno correspondiente a cada una de las cuatro matrices 8×8 .

La variante `ldmatrix.x4.trans` utiliza el mismo mecanismo de carga cooperativa y conserva el mismo orden de las cuatro matrices, pero reorganiza internamente los elementos de cada matriz de forma transpuesta al almacenarlos en los registros. De esta forma, los datos no necesitan encontrarse físicamente transpuestos en memoria compartida: la propia instrucción realiza esta transformación durante la carga.

Esta diferencia resulta especialmente útil al preparar los operandos para las instrucciones MMA. En el kernel CUDA desarrollado, `ldmatrix.x4` se utiliza durante la carga de los fragmentos de K , mientras que `ldmatrix.x4.trans` se emplea para los fragmentos de V . En ambos casos, los registros obtenidos deben organizarse teniendo en cuenta la disposición concreta que espera posteriormente la instrucción matricial.

El uso de `ldmatrix` evita implementar manualmente numerosas cargas y operaciones de intercambio entre threads. Como contrapartida, obliga a conocer con precisión la distribución de los elementos entre los lanes del warp y el orden en el que los cuatro subtiles son depositados en los registros, ya que un error en este mapeo modifica directamente la matriz que finalmente procesan los Tensor Cores.

2.2.4. Swizzling en memoria compartida

La memoria compartida de una GPU NVIDIA se encuentra dividida en bancos que pueden ser accedidos en paralelo; para las arquitecturas tratadas, la documentación CUDA describe la organización y los patrones que originan conflictos [6]. Palabras consecutivas de 32 bits se distribuyen entre bancos consecutivos. Cuando varios threads de un warp necesitan acceder simultáneamente a direcciones diferentes pertenecientes al mismo banco, el acceso debe dividirse en varias transacciones, reduciendo el ancho de banda efectivo.

Este problema resulta especialmente relevante al trabajar con matrices. Aunque una matriz se almacene de forma contigua, el patrón utilizado posteriormente para consumirla puede hacer que numerosos threads accedan simultáneamente a posiciones que terminan perteneciendo a los mismos bancos.

Para evitarlo, el kernel modifica deliberadamente la disposición de los datos al almacenarlos en memoria compartida mediante una técnica denominada *swizzling*. Los datos conservan su organización lógica, pero su posición física dentro de la memoria compartida cambia en función de la fila.

En este trabajo, cada fila de 128 elementos BF16 ocupa 256 bytes y se divide en bloques de 16 bytes, coincidiendo con el tamaño utilizado por las instrucciones `cp.async.cg.16`. El índice físico de cada uno de estos bloques se calcula mediante:

```
swizzled_chunk = logical_chunk ^ (row & 0x7)
```

De esta forma, cada fila aplica una permutación diferente sobre los bloques de 16 bytes. Por ejemplo, para los primeros ocho bloques:

```

row 0: 0 1 2 3 4 5 6 7
row 1: 1 0 3 2 5 4 7 6
row 2: 2 3 0 1 6 7 4 5
row 3: 3 2 1 0 7 6 5 4
...
row 7: 7 6 5 4 3 2 1 0

```

El operador XOR no modifica qué datos contiene cada fila, sino únicamente dónde se almacenan físicamente dentro de ella. Para recuperar posteriormente un elemento se debe aplicar el mismo patrón de transformación al calcular su dirección.

Esta reorganización es especialmente útil debido al tamaño de los bloques utilizados. Cada fragmento de 16 bytes ocupa cuatro bancos consecutivos de memoria compartida. Sin *swizzling*, una fila de 256 bytes tiene un stride que vuelve a situar los mismos chunks sobre los mismos grupos de bancos en filas sucesivas. Por tanto, ciertos patrones de carga matricial pueden hacer que varios threads intenten acceder simultáneamente a los mismos bancos.

Al aplicar el XOR con el índice de fila, cada fila desplaza de forma diferente los grupos de cuatro bancos utilizados por sus chunks. De esta forma, accesos que anteriormente convergían sobre los mismos bancos quedan distribuidos entre distintos grupos, reduciendo o eliminando los *bank conflicts*.

El *swizzling* se aplica únicamente a la representación de los datos en memoria compartida. Los accesos a memoria global continúan realizándose de forma contigua y coalescente; es la dirección de destino de cada `cp.async` la que se modifica antes de escribir el fragmento en memoria compartida. Esto permite mantener un patrón eficiente de acceso a memoria global y, al mismo tiempo, adaptar la representación interna a los patrones de acceso requeridos posteriormente por `ldmatrix` y las instrucciones MMA.

El patrón de *swizzling* constituye, por tanto, una transformación de la disposición física de la matriz cuyo objetivo no es modificar los datos, sino hacer coincidir su organización en memoria compartida con la arquitectura de bancos del hardware y con el patrón de acceso utilizado durante el cálculo.

2.2.5. `mma.sync.aligned.m16n8k16`

La instrucción `mma.sync` permite ejecutar una operación de multiplicación y acumulación matricial utilizando los Tensor Cores de la GPU. La operación realizada puede expresarse como: [7]

$$D = A \times B + C$$

En el kernel desarrollado se utiliza concretamente la variante:

`mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32`

La especificación `m16n8k16` determina las dimensiones de las matrices involucradas en la operación. En este caso:

$$A_{16 \times 16} \times B_{16 \times 8} + C_{16 \times 8} \rightarrow D_{16 \times 8}$$

Por tanto, una única instrucción realiza el producto de un fragmento de 16×16 por otro de 16×8 , acumulando el resultado sobre una matriz de 16×8 . Esto equivale a realizar $16 \times 8 \times 16 = 2048$ multiplicaciones junto con sus correspondientes acumulaciones.

La operación es cooperativa a nivel de warp. Los 32 threads participan conjuntamente en una misma instrucción MMA y cada uno proporciona únicamente una fracción de los operandos almacenada en sus registros. De forma equivalente, el resultado de la operación queda distribuido entre los registros de todos los threads del warp. Ningún thread contiene individualmente las matrices completas.

Los modificadores `row.col` indican la disposición lógica de los operandos: la matriz A se interpreta en formato *row-major*, mientras que B se interpreta en formato *column-major*. Esta disposición es la que determina cómo deben organizarse previamente los fragmentos cargados mediante `ldmatrix`.

Los tipos:

`f32.bf16.bf16.f32`

indican que los operandos A y B utilizan formato BF16, mientras que la acumulación y el resultado se realizan en FP32. De esta forma se aprovecha el mayor rendimiento proporcionado por los Tensor Cores para formatos de menor precisión, manteniendo una mayor precisión durante la acumulación.

El modificador `sync` indica que se trata de una operación cooperativa y síncrona a nivel de warp: los threads participantes ejecutan conjuntamente la misma operación antes de continuar. Por su parte, `aligned` establece que todos los threads del warp deben alcanzar la instrucción de forma convergente; no hace referencia al alineamiento de las direcciones de memoria.

En FlashAttention, esta instrucción constituye el núcleo del cálculo matricial. Se utiliza primero para calcular los fragmentos de QK^T y, posteriormente, para multiplicar las probabilidades obtenidas tras el softmax por los fragmentos de V . Debido a que los Tensor Cores operan sobre tiles de dimensiones fijas, las matrices de mayor tamaño se procesan como una sucesión de estas operaciones `m16n8k16`.

2.3. Herramientas

En esta sección se presentan las principales herramientas, lenguajes y bibliotecas utilizadas durante el desarrollo, compilación, verificación y análisis de rendimiento del proyecto.

2.3.1. C++20

El proyecto requiere un compilador compatible con el estándar C++20. Una de las características utilizadas es `std::bit_cast`, empleada para representar como un `uint32_t` el patrón de bits correspondiente al factor de escala utilizado en el cálculo de la atención.

El motivo de esta implementación es que los parámetros *non-type template* utilizados en el proyecto no pueden representarse directamente mediante valores en coma flotante. Mediante `std::bit_cast` es posible reinterpretar los 32 bits de un `float` como un `uint32_t` sin realizar ninguna conversión numérica. De esta forma, el valor puede utilizarse durante la instanciación del template y, posteriormente, reinterpretarse de nuevo como `float` dentro del kernel. A diferencia de una conversión convencional a entero, este procedimiento conserva exactamente la representación binaria original del número en coma flotante.

2.3.2. CUDA Toolkit 13.0

La versión utilizada durante el desarrollo ha sido CUDA Toolkit 13.0. Este entorno proporciona las herramientas necesarias para compilar y ejecutar código CUDA sobre GPU NVIDIA.

La herramienta principal utilizada es `nvcc`, el compilador de NVIDIA para código CUDA/C++. Además, el toolkit proporciona las cabeceras, bibliotecas y utilidades necesarias para utilizar la API de CUDA y las instrucciones específicas de la arquitectura objetivo.

Durante el desarrollo también se estudió el uso de *shared-memory register spilling*, que permite utilizar memoria compartida como destino para determinados registros derramados en lugar de recurrir directamente a memoria global. Finalmente, esta característica no se utilizó en la implementación final. Solo es compatible con cuda 13.0+

2.3.3. clangd

`clangd` ha sido utilizado como servidor LSP durante el desarrollo del código C++ y CUDA. Proporciona funcionalidades como autocompletado,

navegación entre símbolos, detección de errores y acceso a información sobre tipos y funciones directamente desde el editor.

2.3.4. clang-format

`clang-format` se ha utilizado para mantener un formato consistente en el código C++ y CUDA del proyecto, automatizando aspectos como la indentación, los espacios y la disposición de las diferentes construcciones del lenguaje.

2.3.5. CMake

CMake se utiliza para configurar y gestionar el proceso de compilación del proyecto. Mediante este se compila la extensión nativa utilizada desde PyTorch y los tests desarrollados para comprobar las utilidades implementadas en CUDA y C++. El repositorio incluye un fichero de presets para que la configuración habitual no dependa de introducir manualmente todos los parámetros en cada compilación.

El preset utilizado durante el desarrollo se denomina `angel-wsl`. En él se especifican la arquitectura CUDA objetivo (Ampere, `sm_80`) y las rutas de los compiladores. Estas rutas son necesariamente dependientes del sistema, por lo que el preset se proporciona como una configuración reproducible pero modificable por el usuario. Los productos de compilación se mantienen dentro de `build/`. El Anexo B recoge los comandos necesarios para configurar y compilar el proyecto.

2.3.6. uv

`uv` se utiliza como gestor del entorno y de las dependencias de Python del proyecto. Permite gestionar de forma rápida y reproducible el entorno virtual, las dependencias y la ejecución de las diferentes herramientas escritas en Python.

2.3.7. pytest

`pytest` constituye el entorno principal de pruebas utilizado en la parte Python del proyecto. Se emplea para comprobar el correcto funcionamiento de la extensión C++/CUDA y para verificar la correctitud de las implementaciones realizadas con CUDA y Triton, así como de las llamadas utilizadas como referencia mediante PyTorch SDPA.

2.3.8. Nsight Compute

Nsight Compute se ha utilizado para analizar el rendimiento de las diferentes implementaciones [12]. La herramienta de línea de comandos `ncu` permite obtener métricas detalladas sobre la ejecución de los kernels, incluyendo ocupación, utilización de unidades de cómputo, tráfico de memoria, uso de registros y diferentes tipos de *stalls*.

Los resultados obtenidos pueden almacenarse en informes y visualizarse mediante `ncu-ui`, que proporciona una interfaz gráfica para analizar las métricas recogidas y comparar el comportamiento de distintos kernels.

2.3.9. PyTorch

PyTorch se utiliza tanto como referencia de correctitud como interfaz entre Python y la implementación CUDA. La ruta de atención se basa en `scaled_dot_product_attention` y en el selector explícito de backends `sdpa_kernel` [13]. La función *Scaled Dot Product Attention* (SDPA) sirve como implementación de referencia frente a la que se comparan los resultados obtenidos con CUDA y Triton.

Además, la implementación CUDA se expone a Python mediante una extensión nativa compatible con PyTorch. Esto permite pasar tensores almacenados directamente en la GPU a los kernels desarrollados en C++/CUDA sin necesidad de realizar copias adicionales de los datos.

2.3.10. Triton

Triton es un lenguaje y compilador orientado al desarrollo de kernels para GPU desde Python. Su mecanismo de *autotuning* permite evaluar configuraciones alternativas para una clave determinada antes de reutilizar la configuración seleccionada [15]. Su modelo de programación expresa operaciones sobre bloques de datos y permite parametrizar aspectos como los tamaños de tile, el número de warps y las etapas del pipeline.

En este trabajo se utiliza para implementar una segunda versión de FlashAttention y comparar su complejidad de desarrollo, correctitud y rendimiento con la implementación realizada directamente en CUDA.

2.3.11. Ruff

Ruff se utiliza como herramienta de análisis estático y formato para el código Python del proyecto. Permite detectar errores comunes, problemas

de estilo y construcciones innecesarias, manteniendo una base de código consistente con un coste reducido durante el desarrollo.

2.3.12. GoogleTest

GoogleTest se utiliza como framework de pruebas para el código C++ y CUDA. Mediante estas pruebas se comprueba el comportamiento de diferentes utilidades y componentes de bajo nivel de la implementación antes de integrarlos en el kernel completo.

Su integración con CMake permite compilar y ejecutar las pruebas como parte del proceso normal de construcción del proyecto.

2.3.13. CTest

CTest se utiliza como capa de ejecución para los tests nativos registrados por CMake. De esta forma, las pruebas de C++/CUDA pueden ejecutarse independientemente de `pytest`, manteniendo separados los tests de bajo nivel de las pruebas de integración y rendimiento realizadas desde Python. Esta separación también permite comprobar la parte nativa antes de cargar la extensión desde PyTorch.

2.4. Trabajos relacionados

El principal trabajo relacionado con este proyecto es *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* [16]. En este artículo se introduce FlashAttention como un algoritmo de atención exacta diseñado teniendo en cuenta explícitamente la jerarquía de memoria de las GPU. Su principal aportación consiste en dividir el cálculo en bloques que puedan procesarse utilizando la memoria SRAM disponible en la GPU, evitando materializar completamente la matriz de atención y reduciendo el número de transferencias realizadas hacia memoria global. Este planteamiento constituye la base algorítmica de las implementaciones desarrolladas en este trabajo.

FlashAttention-2 mantiene la formulación exacta y el esquema por bloques, pero introduce cambios específicos de reparto de trabajo: reduce operaciones no matriciales, aumenta el paralelismo entre bloques incluso dentro de una misma cabeza y modifica la partición del trabajo entre *warps* [3]. Esta distinción es especialmente relevante para este TFG, ya que los resultados finales muestran que la granularidad y el reparto del trabajo son tan importantes como las optimizaciones locales del movimiento de datos.

Además del artículo original, existen diferentes trabajos de carácter práctico centrados en la implementación y optimización de FlashAttention directamente sobre GPU. Entre ellos destaca *Writing Speed-of-Light Flash Attention for 5090 in CUDA C++* [17], donde se desarrolla progresivamente un kernel de atención en CUDA C++. Partiendo de una implementación básica, se introducen sucesivamente optimizaciones como *shared-memory swizzling*, *pipelining* y el uso de `ldmatrix.x4`. Aunque el trabajo se ejecuta sobre una NVIDIA RTX 5090, las versiones estudiadas utilizan principalmente mecanismos ya disponibles desde Ampere, como `cp.async`, `ldmatrix` y `mma.sync`, por lo que constituye una referencia especialmente útil para la implementación desarrollada.

Una aproximación similar aparece en la serie *Flash Attention from Scratch* [18]. Este trabajo desarrolla FlashAttention 2 desde cero sobre GPUs NVIDIA Ampere y aplica progresivamente diferentes técnicas de optimización, entre ellas *swizzling*, carga anticipada de los bloques de K y V , solapamiento entre transferencias y cómputo, *double buffering* y optimizaciones a nivel de instrucciones. Resulta especialmente interesante la comparación realizada entre una RTX 3090 y una A100, donde una misma implementación puede presentar comportamientos considerablemente diferentes. Esto pone de manifiesto la dependencia existente entre las técnicas de optimización empleadas y las características concretas del dispositivo sobre el que se ejecuta el kernel.

Gran parte de las técnicas utilizadas en kernels de atención optimizados proceden de las mismas estrategias empleadas en multiplicaciones matriciales de alto rendimiento. Una referencia importante en este ámbito es la documentación de CUTLASS sobre implementación eficiente de GEMM [19]. CUTLASS organiza la multiplicación matricial mediante una jerarquía de tiles a nivel de bloque, warp e instrucción, buscando maximizar la reutilización de los datos en los distintos niveles de la jerarquía de memoria. También emplea *software pipelining* y múltiples buffers para solapar el movimiento de datos con las operaciones matriciales.

En esta misma línea, *How To Write A Fast Matrix Multiplication From Scratch With Tensor Cores* [20] presenta el desarrollo progresivo de un kernel HGEMM utilizando directamente los Tensor Cores. El trabajo parte de una implementación basada en *hierarchical tiling* y aplica sucesivamente técnicas como accesos vectorizados, eliminación de conflictos de bancos mediante *swizzling*, ajuste de las dimensiones de los tiles y *double buffering*. La implementación permite observar de forma práctica cómo el rendimiento de los Tensor Cores está estrechamente relacionado con la capacidad del kernel para alimentar continuamente estas unidades mediante una utilización eficiente de la jerarquía de memoria.

Las dos referencias anteriores son especialmente relevantes para FlashAt-

tention debido a que sus operaciones principales, QK^T y PV , están formadas por multiplicaciones matriciales por bloques. Por tanto, técnicas desarrolladas originalmente para kernels GEMM, como el *tiling*, la reutilización de datos, el *swizzling* o el solapamiento entre transferencia y cómputo, pueden trasladarse directamente a la implementación de atención.

Como referencia complementaria de programación GPU contemporánea se ha utilizado *Modern GPU Programming For ML Sys* [21]. El material presenta de forma progresiva el modelo de ejecución de GPU, disposición de datos, movimiento asíncrono, sincronización y construcción de kernels de alto rendimiento, incluyendo ejemplos de GEMM y FlashAttention. En este trabajo se emplea como referencia de contexto para relacionar las técnicas estudiadas en Ampere con enfoques de programación y planificación utilizados en arquitecturas más recientes.

Finalmente, *Programming Massively Parallel Processors: A Hands-on Approach* [22] se ha utilizado como referencia general para los conceptos relacionados con arquitectura GPU y programación CUDA. El libro presenta la jerarquía de memoria de las GPU, técnicas de optimización de accesos a memoria, multiplicación matricial y diferentes estrategias para incrementar la localidad y el paralelismo de los programas. Aunque no constituye un trabajo específico sobre FlashAttention, proporciona parte de la base conceptual necesaria para comprender las optimizaciones aplicadas en las implementaciones anteriores.

2.5. Contribuciones de este TFG

Las contribuciones de este trabajo no se resumen únicamente en una comparación temporal. Se ha desarrollado y validado una implementación propia de FlashAttention en CUDA, una implementación equivalente en Triton y un banco de pruebas común frente a SDPA y cuDNN. El kernel CUDA está especializado para la configuración estudiada, por lo que los resultados deben interpretarse dentro de ese alcance.

La aportación experimental consiste en relacionar los tiempos medidos con métricas de NVIDIA Nsight Compute. De esta forma se puede distinguir entre optimizaciones locales que producen el efecto esperado —como eliminar accesos no coalescentes o conflictos de banco— y decisiones estructurales, como la granularidad de los CTAs o la reutilización de los tiles de K y V , que terminan condicionando el rendimiento global.

Además de los resultados de rendimiento, el trabajo aporta una implementación reproducible, un test funcional y tests temporales comunes y scripts para automatizar tanto el perfilado como la caracterización del entorno de

ejecución.

Capítulo 3

Metodología de desarrollo

En este capítulo se describe la metodología seguida durante el desarrollo y validación de las distintas implementaciones de FlashAttention. En primer lugar, se presenta el núcleo algorítmico por bloques de FlashAttention y se distingue de las mejoras de particionado introducidas por FlashAttention-2. Posteriormente, se detalla la implementación desarrollada directamente en CUDA, incluyendo las principales decisiones de diseño y optimizaciones aplicadas.

A continuación, se describe la implementación realizada con Triton y el proceso de *autotuning* utilizado para seleccionar sus parámetros de ejecución. Finalmente, se presenta la metodología empleada para verificar la correctitud de los resultados y para detectar y depurar errores durante el desarrollo de los kernels.

Los kernels implementados en CUDA y Triton, así como la implementación de referencia basada en SDPA, se evalúan bajo una configuración común: dimensión de cabeza igual a 128, longitud de secuencia múltiplo de 16, un único elemento por batch y una única cabeza de atención. En todos los casos se emplea atención no causal.

3.1. FlashAttention y FlashAttention-2

El núcleo algorítmico utilizado como punto de partida es FlashAttention: atención exacta por bloques con actualización incremental del *softmax*, de forma que no sea necesario materializar completamente la matriz QK^T en memoria global [16]. El *softmax online* empleado para mantener el máximo y el término de normalización durante el recorrido incremental se apoya en la formulación de Milakov y Gimelshein [4].

FlashAttention-2 conserva este principio, pero añade cambios relevantes

de particionado y paralelización. En particular, reduce trabajo no matricial, paraleliza la atención de una misma cabeza entre varios bloques y modifica la distribución del trabajo entre *warps* para disminuir comunicación y aumentar la utilización de la GPU [3]. La implementación CUDA de este TFG **no pretende reproducir íntegramente el particionado de FlashAttention-2**; utiliza su algoritmo por bloques y varias de sus ideas como referencia, pero adopta un *mapping* propio que posteriormente se analiza de forma experimental.

Por esta razón, el siguiente pseudocódigo debe interpretarse como el **núcleo algorítmico común de FlashAttention**, no como una reproducción completa de todas las decisiones de paralelización de FlashAttention-2.

Algorithm 1 Núcleo algorítmico por bloques de FlashAttention

Require: $Q, K, V \in \mathbb{R}^{N \times d}$
Require: B_r : número de filas por bloque de Q
Require: B_c : número de filas por bloque de K y V
Ensure: $O = \text{softmax}(QK^T/\sqrt{d})V$

- 1: $T_r \leftarrow \lceil N/B_r \rceil$
- 2: $T_c \leftarrow \lceil N/B_c \rceil$
- 3: Dividir Q en bloques $Q_1, \dots, Q_{T_r}$, con $Q_i \in \mathbb{R}^{B_r \times d}$
- 4: Dividir K y V en bloques $K_1, \dots, K_{T_c}$ y $V_1, \dots, V_{T_c}$, con $K_j, V_j \in \mathbb{R}^{B_c \times d}$
- 5: **for** $i \leftarrow 1, \dots, T_r$ **do**
- 6: $O_i \leftarrow 0 \in \mathbb{R}^{B_r \times d}$
- 7: $m_i \leftarrow -\infty \in \mathbb{R}^{B_r}$
- 8: $l_i \leftarrow 0 \in \mathbb{R}^{B_r}$
- 9: **for** $j \leftarrow 1, \dots, T_c$ **do**
- 10: $S_{ij} \leftarrow Q_i K_j^T / \sqrt{d}$
- 11: $m_i^{new} \leftarrow \text{máx}(m_i, \text{rowmax}(S_{ij}))$
- 12: $P_{ij} \leftarrow \exp(S_{ij} - m_i^{new})$
- 13: $\alpha_i \leftarrow \exp(m_i - m_i^{new})$
- 14: $l_i^{new} \leftarrow \alpha_i \odot l_i + \text{rowsum}(P_{ij})$
- 15: $O_i \leftarrow \alpha_i \odot O_i + P_{ij} V_j$
- 16: $m_i \leftarrow m_i^{new}$
- 17: $l_i \leftarrow l_i^{new}$
- 18: **end for**
- 19: $O_i \leftarrow O_i \oslash l_i$
- 20: Escribir O_i en O
- 21: **end for**

En el pseudocódigo anterior, rowmax y rowsum calculan respectivamente

el máximo y la suma de cada fila. Los vectores m_i y l_i almacenan el estado necesario para la normalización incremental. El pseudocódigo expresa dependencias matemáticas; no fija por sí mismo el nivel de la jerarquía de memoria ni el reparto concreto entre bloques y *warps*.

3.1.1. Algoritmo CUDA

La implementación CUDA distribuye el cálculo de la atención a nivel de *warp*. Cada *warp* es responsable de un bloque de 16 filas consecutivas de la matriz de consultas Q , de forma que un bloque CUDA contiene varios *warps* que procesan de manera independiente distintos bloques de consultas. La posición global del *warp* determina las filas de Q y de la salida O que le corresponden, mientras que su identificador dentro del bloque se emplea para cooperar en la transferencia de K y V a memoria compartida. De esta manera, cada *warp* mantiene en registros su bloque de consultas y el acumulador correspondiente a 16 filas de la salida.

Las matrices K y V se recorren en bloques de 16×128 elementos. Para ocultar parcialmente la latencia de memoria global se utiliza un *pipeline* con múltiples buffers en memoria compartida. Mientras un bloque de K y V está siendo consumido por los *warps*, bloques posteriores se transfieren desde memoria global mediante instrucciones asíncronas `cp.async`. Los buffers se organizan de forma circular, de manera que el buffer utilizado para el bloque i puede reutilizarse para almacenar el bloque $i + \text{buffer_num}$ una vez que todos los *warps* han terminado de consumirlo. La carga de cada bloque de K y V se realiza de forma cooperativa entre todos los *warps* del bloque CUDA. Como consecuencia, todos ellos avanzan de forma prácticamente *lockstep* a través del bucle principal: antes de consumir un nuevo bloque deben esperar a que las transferencias asíncronas correspondientes hayan finalizado y sincronizarse mediante una barrera a nivel de bloque. De forma análoga, antes de reutilizar un buffer de memoria compartida, todos los *warps* deben haber terminado de consumir su contenido. Esta sincronización garantiza la corrección del acceso cooperativo a memoria compartida, aunque reduce el grado de independencia temporal entre los distintos *warps* del bloque. La transferencia desde memoria compartida al fichero de registros constituye un segundo nivel de *buffering*; las instrucciones empleadas se corresponden con las primitivas documentadas en la ISA PTX de NVIDIA [7]. Los fragmentos de K y V se cargan mediante `ldmatrix.sync.aligned.m8n8.x4.shared.b16`, que permite obtener simultáneamente cuatro matrices 8×8 desde memoria compartida y distribuirlas directamente entre los registros del *warp*. El código mantiene un pequeño *pipeline* entre estas cargas y las operaciones matriciales: mientras un par de fragmentos se utiliza en los Tensor Cores, se carga desde

memoria compartida el siguiente par que será consumido.

Los productos matriciales se realizan mediante instrucciones `mma.sync` de tipo $m16n8k16$, con operandos BF16 y acumulación FP32. Cada instrucción calcula un producto 16×16 por 16×8 , produciendo un fragmento 16×8 en precisión simple. Para calcular QK^T , un bloque lógico de puntuaciones de tamaño 16×16 se representa mediante dos fragmentos 16×8 . Tras aplicar el escalado y el *softmax* online, las probabilidades se convierten nuevamente a BF16 y se utilizan como operando izquierdo en los productos PV . El acumulador de salida permanece en FP32 durante todo el recorrido de K y V , realizándose la conversión a BF16 únicamente al escribir el resultado final. Para mejorar el acceso posterior mediante `ldmatrix`, los bloques de K y V se almacenan en memoria compartida utilizando un patrón *swizzled* con granularidad de ocho elementos BF16 (16 bytes). La transformación aplica un XOR entre la posición lógica del segmento y los bits bajos de la fila, redistribuyendo los accesos entre los bancos de memoria compartida. Al cargar posteriormente los fragmentos mediante `ldmatrix`, se aplica la misma transformación para reconstruir la dirección física correspondiente. De este modo, la disposición en memoria global puede mantenerse contigua y coalescente mientras la disposición en memoria compartida se adapta al patrón de acceso de las cargas matriciales.

En conjunto, el kernel emplea dos niveles explícitos de movimiento de datos: un *pipeline* memoria global–memoria compartida mediante `cp.async`, y un segundo *pipeline* memoria compartida–registros mediante `ldmatrix`. Sobre los fragmentos residentes en registros se ejecutan las instrucciones `mma.sync` de los Tensor Cores, manteniendo tanto los estados del *softmax* online como el acumulador de salida en registros durante toda la operación.

Algorithm 2 FlashAttention CUDA

```
1: Asignar a cada warp 16 filas consecutivas de  $Q$ 
2: Cargar el bloque de  $Q$  correspondiente en registros
3: Inicializar acumulador  $O \leftarrow 0$ , máximo de fila  $m \leftarrow -\infty$  y denominador  $l \leftarrow 0$ 
4: for  $b \leftarrow 0$  hasta buffer_num - 1 do
5:   Cargar cooperativamente  $K_b$  y  $V_b$  de memoria global a memoria compartida mediante cp.async
6:   Aplicar swizzle XOR durante la escritura en memoria compartida
7: end for
8: for cada bloque  $i$  de 16 filas de  $K$  y  $V$  do
9:   Esperar al buffer correspondiente mediante cp.async.wait_group
10:  Sincronizar todos los warps del bloque  $\triangleright$  Los warps avanzan prácticamente en lockstep
11:   $S \leftarrow 0$ 
12:  Cargar primeros fragmentos de  $K_i$  desde memoria compartida a registros con ldmatrix
13:  for cada fragmento de dimensión  $16 \times 16$  en la dimensión de cabeza do
14:    Cargar anticipadamente el siguiente fragmento de  $K_i$  mediante ldmatrix
15:     $S \leftarrow S + QK_i^T$  mediante mma.sync.m16n8k16
16:  end for
17:   $S \leftarrow S/\sqrt{d}$ 
18:   $m_{\text{new}} \leftarrow \text{máx}(m, \text{rowmax}(S))$ 
19:   $\alpha \leftarrow \exp(m - m_{\text{new}})$ 
20:   $P \leftarrow \exp(S - m_{\text{new}})$ 
21:   $l_{\text{new}} \leftarrow \alpha l + \text{rowsum}(P)$ 
22:  Convertir  $P$  de FP32 a BF16 para utilizarlo como operando de los Tensor Cores
23:   $O \leftarrow \alpha O$ 
24:  Cargar primeros fragmentos de  $V_i$  desde memoria compartida a registros mediante ldmatrix
25:  for cada fragmento  $16 \times 8$  de  $V_i$  do
26:    Cargar anticipadamente el siguiente fragmento de  $V_i$ 
27:     $O \leftarrow O + PV_i$  mediante mma.sync.m16n8k16
28:  end for
29:   $m \leftarrow m_{\text{new}}$ 
30:   $l \leftarrow l_{\text{new}}$ 
31:  Sincronizar todos los warps antes de reutilizar el buffer
32:  if existen bloques futuros de  $K$  y  $V$  then
33:    Reutilizar el buffer consumido
34:    Cargar asíncronamente el siguiente bloque  $K_{i+\text{buffer\_num}}, V_{i+\text{buffer\_num}}$ 
35:  end if
36: end for
37:  $O \leftarrow O/l$ 
38: Redistribuir los elementos de  $O$  entre los hilos del warp
39: Escribir  $O$  en memoria global mediante stores coalescentes
```

3.1.2. FlashAttention: Triton

La implementación en Triton sigue la misma descomposición algorítmica que las versiones anteriores. Cada instancia del programa Triton procesa un bloque de B_r filas consecutivas de la matriz de consultas Q y recorre secuencialmente la dimensión de secuencia de K y V en bloques de B_c filas. De esta forma, cada programa mantiene localmente el acumulador de salida y los parámetros necesarios para calcular el *softmax online*, evitando materializar la matriz completa QK^T en memoria global.

Dado que la dimensión de cabeza se fija en $d = 128$, cada bloque de consultas tiene dimensiones $B_r \times 128$, mientras que los bloques de claves y valores tienen dimensiones $B_c \times 128$. Para cada bloque de K se calcula una matriz temporal de puntuaciones de dimensiones $B_r \times B_c$. Esta matriz únicamente permanece durante la iteración actual del algoritmo y se utiliza inmediatamente para actualizar el *softmax* y el acumulador de salida.

El producto matricial se expresa mediante la operación `tl.dot`. Con operandos BF16 sobre arquitecturas que disponen de Tensor Cores, el compilador de Triton genera las instrucciones matriciales correspondientes y mantiene la acumulación en FP32.

La implementación se parametriza mediante cuatro parámetros principales:

- B_r : número de filas de Q procesadas por cada instancia del programa.
- B_c : número de filas de K y V procesadas en cada iteración.
- `num_warps`: número de *warps* utilizados para ejecutar una instancia del programa.
- `num_stages`: número de etapas utilizadas por el *pipeline* de cargas y cómputo generado por Triton.

La dimensión de cabeza no forma parte del espacio de búsqueda, ya que se mantiene fija en 128 para todos los experimentos. De forma análoga, el tamaño de batch y el número de cabezas se fijan ambos a uno. Por tanto, el proceso de *autotuning* explora únicamente combinaciones de los cuatro parámetros anteriores.

El valor de B_r determina la cantidad de filas de salida calculadas simultáneamente y, por tanto, afecta directamente al tamaño del acumulador mantenido por cada programa. El valor de B_c determina el tamaño de los bloques temporales de puntuaciones y la granularidad con la que se recorren K y V . Valores elevados de ambos parámetros pueden incrementar la reutilización de datos y el trabajo realizado por programa, pero también aumentan el consumo de registros y memoria compartida. Por su parte, `num_warps`

controla el paralelismo dentro de cada programa, mientras que `num_stages` modifica la profundidad del *pipeline* utilizado para solapar transferencias de memoria y cómputo.

Las distintas combinaciones se evalúan mediante el mecanismo de *autotuning* de Triton, seleccionándose para cada configuración aquella que presenta el menor tiempo de ejecución. De este modo se exploran automáticamente distintas relaciones entre tamaño de bloque, paralelismo y profundidad del *pipeline*.

El funcionamiento del kernel puede resumirse mediante el Algoritmo 3.

Algorithm 3 FlashAttention Triton

Require: $Q, K, V \in \mathbb{R}^{N \times 128}$

Require: B_r : filas de Q procesadas por programa

Require: B_c : filas de K y V procesadas por iteración

Ensure: $O = \text{softmax}(QK^T/\sqrt{128})V$

- 1: Asignar a cada programa un bloque $Q_i \in \mathbb{R}^{B_r \times 128}$
 - 2: Cargar Q_i
 - 3: $O_i \leftarrow 0 \in \mathbb{R}^{B_r \times 128}$ en FP32
 - 4: $m_i \leftarrow -\infty \in \mathbb{R}^{B_r}$
 - 5: $l_i \leftarrow 0 \in \mathbb{R}^{B_r}$
 - 6: **for** cada bloque $K_j, V_j \in \mathbb{R}^{B_c \times 128}$ **do**
 - 7: Cargar K_j
 - 8: $S_{ij} \leftarrow \text{tl.dot}(Q_i, K_j^T)/\sqrt{128}$
 - 9: $m_i^{new} \leftarrow \max(m_i, \text{rowmax}(S_{ij}))$
 - 10: $\alpha_i \leftarrow \exp(m_i - m_i^{new})$
 - 11: $P_{ij} \leftarrow \exp(S_{ij} - m_i^{new})$
 - 12: $l_i^{new} \leftarrow \alpha_i \odot l_i + \text{rowsum}(P_{ij})$
 - 13: $O_i \leftarrow \alpha_i \odot O_i$
 - 14: Cargar V_j
 - 15: $O_i \leftarrow O_i + \text{tl.dot}(P_{ij}, V_j)$
 - 16: $m_i \leftarrow m_i^{new}$
 - 17: $l_i \leftarrow l_i^{new}$
 - 18: **end for**
 - 19: $O_i \leftarrow O_i \oslash l_i$
 - 20: Convertir O_i al formato de salida
 - 21: Escribir O_i en memoria global
-

El pseudocódigo describe el flujo algorítmico del programa Triton; la planificación concreta de las operaciones y del movimiento de datos depende del código generado para la configuración seleccionada.

Capítulo 4

Entorno de prueba

En este capítulo se describe el entorno hardware y software utilizado para la evaluación experimental de las distintas implementaciones de FlashAttention. Todas las mediciones presentadas en los capítulos posteriores se han realizado sobre la misma plataforma, con el objetivo de que las comparaciones entre CUDA, Triton y las implementaciones proporcionadas por PyTorch se efectúen bajo condiciones equivalentes.

El entorno de pruebas corresponde a una máquina virtual proporcionada a través de la plataforma Vast.ai. Esta plataforma permite acceder de forma remota a sistemas equipados con GPUs de altas prestaciones. En concreto, para los experimentos se seleccionó una instancia equipada con una GPU NVIDIA A100-SXM4 de 40 GB. Al tratarse de un entorno virtualizado, algunos parámetros de bajo nivel del sistema anfitrión, como el estado de *boost* del procesador, no son accesibles desde la máquina virtual.

4.1. Hardware

Los experimentos se han ejecutado sobre una GPU NVIDIA A100-SXM4 con 40 GB de memoria. Esta GPU pertenece a la arquitectura Ampere y presenta capacidad de cómputo 8.0 (`sm_80`). El dispositivo dispone de 108 *Streaming Multiprocessors* (SM) y está específicamente orientado a cargas de trabajo de cómputo de alto rendimiento.

La Tabla 4.1 resume las principales características del dispositivo empleado.

Cuadro 4.1: Características de la GPU utilizada durante los experimentos.

Característica	Valor
Proveedor de la instancia	Vast.ai
GPU	NVIDIA A100-SXM4-40GB
Arquitectura	Ampere
Capacidad de cómputo	8.0 (<code>sm_80</code>)
Número de SM	108
Memoria	40 GB
Límite de potencia	400 W
Frecuencia máxima de los SM	1410 MHz
Frecuencia de memoria	1215 MHz

La máquina virtual dispone además de recursos de CPU pertenecientes a un procesador AMD EPYC 7343. La instancia utilizada expone 7 CPU lógicas al sistema invitado, con una frecuencia observada de aproximadamente 3.19 GHz. La máquina dispone de aproximadamente 49 GiB de memoria principal. Debido a la virtualización, la información relativa al estado de *boost* del procesador no es expuesta por el sistema y, por tanto, no se controla explícitamente durante los experimentos.

4.2. Entorno software

El sistema operativo utilizado es Ubuntu 22.04.5 LTS, con kernel Linux 6.8.0-138-generic y arquitectura x86-64.

Las implementaciones CUDA se han compilado utilizando CUDA Toolkit 13.0 y `nvcc` 13.0.88. El código C++ auxiliar se compila mediante GCC 11.4.0 y la generación del proyecto se realiza con CMake 4.4.3.

La Tabla 4.2 recoge las principales versiones utilizadas.

Cuadro 4.2: Entorno software utilizado durante los experimentos.

Componente	Versión
Ubuntu	22.04.5 LTS
Kernel Linux	6.8.0-138-generic
Driver NVIDIA	580.173.02
CUDA Toolkit	13.0
<code>nvcc</code>	13.0.88
GCC / G++	11.4.0
CMake	4.4.3
Python	3.14.7
PyTorch	2.13.0+cu130
CUDA de PyTorch	13.0
cuDNN	9.2.0
Triton	3.7.1
Nsight Compute	2025.3.1

PyTorch detecta correctamente la GPU como una A100 con capacidad de cómputo 8.0. Tanto la implementación Triton como las implementaciones de referencia basadas en SDPA se ejecutan dentro del mismo entorno de Python, evitando diferencias derivadas de utilizar distintas versiones del *runtime* de CUDA.

4.3. Metodología de medida

Las distintas implementaciones se evalúan utilizando las mismas dimensiones de entrada, el mismo tipo de datos y la misma política de generación de entradas. Se considera un único elemento por *batch*, una única cabeza de atención, dimensión de cabeza $d = 128$ y atención no causal. Los tensores Q , K y V utilizan formato BF16.

Todos los tests de rendimiento utilizan la misma semilla pseudoaleatoria, fijada a 0 mediante `torch.manual_seed(0)` y `torch.cuda.manual_seed_all(0)` antes de generar las entradas. De esta forma, CUDA, Triton, SDPA y cuDNN parten de entradas reproducibles construidas bajo el mismo criterio para una misma longitud de secuencia. Esta decisión evita introducir una fuente de variación innecesaria al comparar implementaciones.

Para las medidas temporales se realiza inicialmente una ejecución de calentamiento. Esta primera ejecución permite excluir operaciones que ocurren únicamente al comienzo de la prueba, como la compilación JIT de los kernels Triton o la inicialización de determinados componentes del entorno CUDA.

Posteriormente, cada configuración se ejecuta 100 veces. El tiempo de ejecución se obtiene mediante eventos CUDA, evitando incluir en la medida el tiempo empleado por Python o por otras operaciones realizadas en CPU. Para cada longitud de secuencia se calcula la media y la desviación típica de los tiempos obtenidos. Las cuatro rutas se evalúan con el mismo conjunto de longitudes de secuencia y el mismo número de repeticiones.

Las dos rutas de PyTorch se fuerzan de forma explícita mediante el contexto `torch.nn.attention.sdpa_kernel`: una ejecución selecciona `SDPBackend.FLASH_ATTENTION` y la otra `SDPBackend.CUDNN_ATTENTION`. De este modo, el benchmark no depende de la selección automática del backend realizada por PyTorch [13]. En Triton, la primera llamada asociada a una configuración se realiza fuera de la región temporizada; así, la compilación JIT y las evaluaciones necesarias para el *autotuning* no forman parte de las 100 repeticiones medidas [15].

La instancia de Vast.ai no fija manualmente las frecuencias de GPU. Por tanto, las medidas pueden estar sometidas a la política dinámica de frecuencia y a cierta variabilidad propia de un entorno virtualizado. Para reducir su impacto, todas las implementaciones se miden en la misma instancia y sesión experimental y se reportan media y desviación típica. Las desviaciones observadas son pequeñas respecto a las diferencias entre implementaciones, pero esta condición se conserva como limitación de validez externa.

Para el análisis detallado de los kernels se utiliza NVIDIA Nsight Compute 2025.3.1 [12]. Los tests de perfilado realizan un calentamiento previo y delimitan mediante NVTX una única ejecución del kernel de interés. La instrumentación de Nsight Compute puede introducir *replay* y alterar la duración observada, por lo que los tiempos del perfilador no se utilizan como benchmark principal. Las métricas recogidas se emplean únicamente para explicar la ocupación, el tráfico de memoria, el uso de registros y memoria compartida, la actividad de Tensor Cores y las causas de espera de los *warps*.

Capítulo 5

Tests

Durante el desarrollo se utilizó un test funcional para comprobar la corrección numérica y un conjunto separado de tests no funcionales para estudiar el rendimiento mediante medidas temporales y herramientas de *profiling*.

Los tests se implementan utilizando `pytest`. Las pruebas correspondientes a CUDA, Triton y las implementaciones de referencia de PyTorch se mantienen separadas, permitiendo ejecutar de forma independiente las diferentes familias de tests.

5.1. Test funcional

La validación funcional se concentra en una única prueba de referencia. Para cada implementación se generan tensores pseudoaleatorios Q , K y V en BF16 y se compara la salida obtenida con la producida por `scaled_dot_product_attention` de PyTorch bajo la misma configuración.

Para garantizar la reproducibilidad se fija la semilla a 0 antes de generar las entradas. La comparación se realiza mediante `torch.allclose`, utilizando tolerancias relativa y absoluta de $2 \cdot 10^{-2}$. No se exige igualdad bit a bit porque las implementaciones pueden ejecutar las operaciones en distinto orden y utilizar BF16 en puntos diferentes del cálculo.

Este test se ejecuta después de los cambios relevantes del kernel y actúa como comprobación automática de que las optimizaciones no alteran el resultado numérico esperado. Se utiliza una única semilla y una tolerancia fija; por tanto, la validación demuestra compatibilidad numérica para los casos ejercitados, pero no constituye una caracterización exhaustiva del error numérico. El objetivo del trabajo no es estudiar estrategias de validación de atención, por lo que no se incorporan campañas multi-semilla ni métricas

específicas de error máximo o medio.

5.2. Tests no funcionales

Los tests no funcionales no verifican directamente el valor de la salida, sino que se utilizan para caracterizar el rendimiento de las diferentes implementaciones. Se distinguen dos tipos: tests de tiempo y tests destinados al *profiling*.

5.2.1. Tests de tiempo

Los tests de tiempo permiten estudiar cómo varía el coste de la operación al aumentar la longitud de secuencia. Se utilizan las mismas dimensiones y tipos de datos para CUDA, Triton y SDPA, de forma que los resultados puedan compararse directamente.

Todos los tests temporales inicializan los generadores pseudoaleatorios con la misma semilla, 0, antes de generar las entradas. Este criterio se mantiene para CUDA, Triton, SDPA y cuDNN.

Para cada longitud de secuencia se generan los tensores Q , K y V en BF16. Antes de comenzar la medida se realiza al menos una ejecución de calentamiento. Esta ejecución es especialmente importante para Triton, ya que permite excluir de la medida el tiempo correspondiente a la compilación JIT del kernel.

El tiempo se mide utilizando eventos CUDA. Para cada repetición se registra un evento inmediatamente antes del lanzamiento de la implementación y otro inmediatamente después. De esta forma, la medida corresponde al trabajo realizado en la GPU y no incluye el tiempo empleado por Python en preparar la llamada.

Cada configuración se ejecuta múltiples veces. Si los tiempos obtenidos son

$$t_1, t_2, \dots, t_R,$$

el tiempo utilizado como resultado es la media

$$\bar{t} = \frac{1}{R} \sum_{i=1}^R t_i,$$

acompañada de su desviación típica

$$\sigma = \sqrt{\frac{1}{R-1} \sum_{i=1}^R (t_i - \bar{t})^2}.$$

Por tanto, para cada implementación se obtiene un conjunto de valores de la forma

$$(N, \bar{t}, \sigma),$$

que permite estudiar tanto el rendimiento medio como la variabilidad de las mediciones al incrementar la longitud de secuencia.

Los tests destinados a estas medidas se identifican mediante un marcador específico de `pytest`, lo que permite ejecutarlos independientemente del test funcional. De esta manera, las pruebas de rendimiento pueden lanzarse mediante una única batería común para CUDA, Triton y SDPA.

5.2.2. Tests de profiling

Para estudiar las causas de las diferencias de rendimiento se utilizan tests específicos de *profiling*. A diferencia de los tests temporales, estos tests realizan únicamente las ejecuciones necesarias para que una herramienta externa pueda recoger métricas del kernel.

Antes de la ejecución analizada se realiza una llamada de calentamiento seguida de una sincronización de CUDA. Posteriormente se lanza una única ejecución del kernel de interés. La región correspondiente se delimita mediante anotaciones NVTX, lo que permite identificarla desde herramientas de análisis externas y evitar la captura de las operaciones utilizadas para preparar los tensores o realizar el calentamiento.

Los perfiles se obtienen utilizando NVIDIA Nsight Compute. Entre las métricas estudiadas se encuentran la ocupación de los SM, el número de *warps* activos y elegibles, la utilización de los Tensor Cores, el tráfico a través de memoria global y caché L2, y las principales causas de espera de los *warps*.

En particular, estas pruebas permiten diferenciar situaciones en las que el rendimiento está limitado por el ancho de banda de memoria de aquellas en las que el principal problema es la falta de paralelismo disponible para ocultar la latencia. También permiten analizar el efecto sobre el rendimiento de parámetros de implementación como el número de *warps* por bloque, el número de buffers utilizados en memoria compartida o el consumo de registros.

Los tests de *profiling* se mantienen separados de los tests temporales. Esto es necesario debido a que la instrumentación realizada por Nsight Compute puede requerir múltiples ejecuciones o *replays* del kernel y, por tanto, los

tiempos registrados durante una sesión de *profiling* no se utilizan como medidas directas de rendimiento.

5.3. Organización de las pruebas

La separación entre validación funcional y pruebas de rendimiento permite utilizar cada conjunto con un objetivo diferente durante el desarrollo. El test funcional se ejecuta después de realizar modificaciones en los kernels para comprobar que se mantiene la correctitud, mientras que los tests temporales y de *profiling* se utilizan posteriormente para determinar si dichas modificaciones producen una mejora real de rendimiento.

Esta metodología permite que una optimización sea considerada válida únicamente cuando mantiene la correctitud numérica y produce una mejora medible bajo las mismas condiciones experimentales.

Capítulo 6

Análisis de los resultados

En este capítulo se comparan las cuatro rutas evaluadas: la implementación propia en CUDA, la implementación Triton, SDPA y SDPA utilizando el backend de cuDNN. Las métricas de Nsight Compute corresponden a la configuración de perfilado con $N = 8192$, $d = 128$, BF16, $B = H = 1$ y atención no causal. Las medidas temporales se realizan de forma independiente mediante eventos CUDA y 100 repeticiones por longitud de secuencia, utilizando la misma semilla pseudoaleatoria en todas las implementaciones.

El análisis distingue deliberadamente entre **hechos medidos** y **hipótesis de interpretación**. Las métricas permiten observar qué ocurre durante la ejecución; cuando se propone una causa que no está demostrada directamente por un contador hardware, se presenta explícitamente como interpretación o línea de investigación.

6.1. Kernels observados en SDPA

En la configuración perfilada, la ruta SDPA que no fuerza cuDNN ejecuta dos kernels principales:

```
flash_fwd_splitkv_kernel  
flash_fwd_splitkv_combine_kernel
```

La presencia de ambos kernels es un **hecho observado** directamente en Nsight Compute. El nombre del primero indica que la implementación utiliza una variante denominada *split-KV*; sin embargo, este trabajo no analiza ni reconstruye el mecanismo interno empleado por PyTorch para realizar esa partición. Para la comparación experimental sólo se retienen dos observaciones directamente medibles: SDPA lanza un número mayor de CTAs que las otras rutas y necesita un segundo kernel de combinación. Cuando se comparan

tiempos, el coste de la ruta SDPA debe entenderse como el coste conjunto de los kernels que forman la operación.

6.2. Registros y ocupación

Las Tablas 6.1 y 6.2 separan el consumo de registros de la ocupación para mantener una tipografía legible. El número de *warps* por CTA y la residencia máxima se recogen posteriormente junto con la granularidad de lanzamiento. CUDA presenta el menor número de registros por hilo y la mayor ocupación teórica, pero obtiene la menor ocupación alcanzada.

Cuadro 6.1: Uso de registros por hilo para $N = 8192$.

Impl.	Reg./hilo	Reg. asignados
CUDA	148	152
Triton	199	200
SDPA	240	240
cuDNN	209	216

Cuadro 6.2: Ocupación teórica y alcanzada para $N = 8192$.

Impl.	Ocup. teórica	Ocup. alcanzada
CUDA	18.75 %	7.56 %
Triton	12.50 %	12.50 %
SDPA	12.50 %	11.51 %
cuDNN	12.50 %	12.50 %

La Tabla 6.2 muestra que una mayor ocupación teórica no implica automáticamente mayor rendimiento. CUDA podría mantener hasta tres CTAs residentes por SM, pero el perfil registra sólo un 7.56 % de ocupación alcanzada, equivalente a aproximadamente 4.84 *warps* activos por SM.

La ocupación alcanzada no se obtiene simplemente dividiendo el número de CTAs lanzados entre el número de SM. Nsight Compute la calcula a partir de la actividad efectiva de los *warps*. No obstante, el tamaño del *grid* explica por qué CUDA no puede aprovechar la residencia teórica disponible. Para $N = 8192$ se lanzan 128 CTAs sobre 108 SM:

$$\frac{128}{108} \simeq 1,19 \text{ CTA/SM.} \quad (6.1)$$

Por tanto, aunque los recursos permitirían mantener tres CTAs simultáneos, no existe suficiente trabajo en el *grid* para proporcionar tres bloques a cada SM. La Figura 6.1 ilustra esta diferencia entre capacidad de residencia y trabajo realmente disponible.



Figura 6.1: Residencia permitida por recursos frente a trabajo disponible para el kernel CUDA.

Triton y el kernel cuDNN perfilado utilizan ocho *warps* por CTA. Aunque sus recursos sólo permiten un CTA residente, ese único bloque ya proporciona ocho *warps* al SM y ambos alcanzan su ocupación teórica. Esta afirmación describe únicamente la configuración observada en el lanzamiento; no reconstruye el *mapping* interno del kernel cuDNN. SDPA utiliza cuatro *warps* por CTA, como CUDA, pero el perfil muestra un *grid* de 384 CTAs y una ocupación alcanzada próxima al máximo permitido por sus recursos.

6.3. Memoria compartida y granularidad de lanzamiento

La Tabla 6.3 muestra que el kernel CUDA tampoco está limitado por un consumo excesivo de memoria compartida. De hecho, utiliza menos memoria compartida por CTA que las demás implementaciones.

Cuadro 6.3: Memoria compartida y granularidad de lanzamiento para $N = 8192$.

Impl.	SMEM/CTA	CTAs	Hilos/CTA	Warps/CTA
CUDA	32 KiB	128	128	4
Triton	96 KiB	64	256	8
SDPA	80 KiB	384	128	4
cuDNN	66 KiB aprox.	64	256	8

En CUDA, los cuatro buffers circulares de K y V ocupan $4 \times 8 \text{ KiB} = 32 \text{ KiB}$ por CTA. El perfil puede mostrar además una pequeña cantidad de memoria compartida reservada por el sistema. Los datos de la Tabla 6.3 refuerzan la conclusión anterior: el kernel propio libera recursos suficientes

para alojar más CTAs, pero el *grid* no contiene bloques suficientes para explotar esa ventaja en $N = 8192$.

6.4. Conflictos de banco

La Tabla 6.4 recoge las métricas de memoria compartida. CUDA y cuDNN no presentan conflictos de banco. En CUDA, este resultado confirma que el patrón de *swizzling* implementado cumple su objetivo.

Cuadro 6.4: Conflictos de banco observados en los kernels principales.

Impl.	Wavefronts	Conflictos	Conflictos/wavefronts
CUDA	20 971 520	0	0 %
Triton	21 053 440	5 245	0.025 %
SDPA	22 265 856	23 284	0.105 %
cuDNN	18 989 056	0	0 %

Los conflictos de Triton y SDPA representan una fracción muy pequeña del total de *wavefronts* y no generan *wavefronts* excesivos en estas mediciones. Por ello, no constituyen una explicación convincente de las diferencias de tiempo.

Sí resulta interesante que implementaciones muy optimizadas acepten un número no nulo de conflictos. Una **hipótesis plausible**, pero no demostrable únicamente con estos contadores, es que eliminar los últimos conflictos podría exigir una disposición de datos, direccionamiento o presión de registros cuyo coste fuese superior a la penalización evitada. Esta observación se mantiene como línea de análisis y no como conclusión atribuida al compilador.

6.5. Coalescencia de los accesos globales

La Tabla 6.5 compara los sectores globales solicitados con el número ideal calculado por Nsight Compute. Los cuatro kernels principales presentan el mismo número de sectores reales e ideales.

Cuadro 6.5: Sectores globales reales, ideales y excesivos.

Impl.	Sectores	Sectores ideales	Excesivos
CUDA	16 908 288	16 908 288	0
Triton	8 519 680	8 519 680	0
SDPA	17 370 112	17 370 112	0
cuDNN	8 519 632	8 519 632	0

Por tanto, la coalescencia no explica el orden de rendimiento. En particular, el kernel CUDA no pierde rendimiento por solicitar sectores adicionales debidos a accesos globales mal agrupados. Los sectores analizados en este apartado corresponden a tráfico de datos generado por instrucciones de memoria global; el comportamiento del *frontend* y de la caché de instrucciones se estudia mediante métricas diferentes.

6.6. Número total de sectores solicitados

Aunque todos los kernels sean coalescentes, la cantidad total de sectores solicitados es distinta. La Tabla 6.6 normaliza el tráfico respecto a Triton.

Cuadro 6.6: Número total de sectores globales solicitados.

Impl.	Sectores	Respecto a Triton
Triton	8 519 680	1.00×
cuDNN	8 519 632	1.00×
CUDA	16 908 288	1.98×
SDPA	17 370 112	2.04×

Aparecen dos grupos muy claros: Triton y cuDNN solicitan unos 8.52 millones de sectores, mientras que CUDA y SDPA se sitúan alrededor de 17 millones. Este resultado no significa que CUDA acceda peor a memoria; significa que realiza aproximadamente el doble de solicitudes de datos a lo largo de la ejecución. La documentación de cuDNN describe su operación SDPA como una implementación basada en FlashAttention-2 [14], pero este trabajo no presupone a partir de ello un *mapping* interno concreto.

La implementación propia procesa 16 filas de Q por *warp* y cuatro *warps* por CTA, es decir, 64 filas de Q por bloque. El mapping observado para Triton cubre aproximadamente 128 filas de Q por CTA. La Figura 6.2 muestra la interpretación utilizada en este trabajo: un bloque mayor de consultas permite amortizar cada tile de K, V sobre más filas de salida.

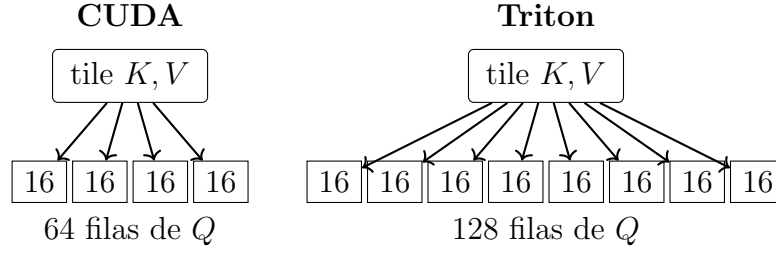


Figura 6.2: Interpretación conceptual de la reutilización de un tile de K, V sobre las filas de consultas procesadas por un CTA.

La correspondencia entre un tile mayor de Q y una mayor reutilización de K, V es una **interpretación consistente con las métricas**, no una demostración de que todo el factor dos proceda exclusivamente de esa causa. No obstante, la coincidencia entre el factor de dos en filas de Q por CTA y el factor aproximado de dos en sectores constituye una evidencia fuerte de que la granularidad global del tile es relevante.

El caso de SDPA es especialmente instructivo: solicita incluso más sectores que CUDA y, aun así, es la ruta más rápida. Por tanto, el número total de sectores no explica por sí solo el rendimiento. El perfil muestra además un número de CTAs considerablemente mayor, por lo que la comparación debe considerar conjuntamente tráfico, reutilización y cantidad de trabajo disponible.

El número de sectores tampoco puede atribuirse al uso de `#pragma unroll`. El desenrollado afecta principalmente al código generado y puede aumentar la presión sobre el *frontend* o la caché de instrucciones, pero no convierte por sí solo 8.5 millones de sectores de datos en 16.9 millones.

6.7. Actividad de Tensor Cores y jerarquía de memoria

La Tabla 6.7 resume varias señales adicionales del perfil. Los porcentajes de actividad Tensor indican que todas las rutas utilizan las unidades matriciales, pero con grados muy diferentes de continuidad.

Cuadro 6.7: Actividad aproximada del pipeline Tensor y señales de memoria para $N = 8192$.

Impl.	Tensor activo	DRAM pico	L2 hit rate
CUDA	18.1 %	0.52 %	97.98 %
Triton	26.4 %	0.75 %	95.86 %
SDPA	57.8 %	1.73 %	97.82 %
cuDNN	38.3 %	1.09 %	95.86 %

Como se observa en la Tabla 6.7, ninguna de las implementaciones se aproxima a saturar el ancho de banda de HBM en este perfil y todas presentan una tasa alta de aciertos en L2. Por ello, los datos no apoyan la hipótesis de que el kernel propio sea simplemente un kernel limitado por ancho de banda de DRAM.

La baja actividad Tensor de CUDA indica, en cambio, que las instrucciones `mma.sync` no se emiten de forma continua. La estructura del kernel ofrece varias causas compatibles con esta observación: actualizaciones frecuentes del *softmax online*, reducciones escalares, dependencias entre resultados, cargas desde memoria compartida y barreras entre los cuatro *warps* del CTA. Con $B_c = 16$, una secuencia de 8192 elementos requiere

$$\frac{8192}{16} = 512 \quad (6.2)$$

iteraciones del bucle principal, cada una con cálculo matricial, actualización de normalización y coordinación del pipeline. Se considera por tanto una **hipótesis respaldada por la estructura del kernel** que parte de la diferencia restante provenga de la dificultad para mantener alimentados los Tensor Cores entre grupos de MMA, especialmente cuando hay pocos *warps* independientes disponibles.

6.8. Mediciones temporales

Las Tablas 6.8 y 6.9 recogen las medias y desviaciones típicas obtenidas mediante eventos CUDA. Todas las implementaciones utilizan la misma semilla 0, las mismas dimensiones, BF16 y 100 repeticiones por longitud de secuencia. Se presentan en dos tablas para mantener una tipografía legible.

Cuadro 6.8: Tiempo medio y desviación típica de CUDA y Triton (ms).

N	CUDA	Triton
8 192	0.579 ± 0.002	0.467 ± 0.007
16 384	1.633 ± 0.032	1.702 ± 0.004
32 768	5.654 ± 0.061	5.235 ± 0.028
65 536	22.243 ± 0.185	18.122 ± 0.091
131 072	85.192 ± 0.279	72.471 ± 0.356
262 144	341.337 ± 1.500	278.601 ± 0.054
524 288	1380.594 ± 7.278	1113.618 ± 0.221
1 048 576	5576.997 ± 15.101	4453.727 ± 3.102

Cuadro 6.9: Tiempo medio y desviación típica de SDPA y cuDNN (ms).

N	SDPA	cuDNN
8 192	0.252 ± 0.020	0.317 ± 0.005
16 384	1.163 ± 0.018	1.204 ± 0.021
32 768	3.783 ± 0.071	3.840 ± 0.108
65 536	13.148 ± 0.116	13.506 ± 0.213
131 072	52.172 ± 0.299	54.304 ± 0.312
262 144	199.515 ± 0.411	207.784 ± 0.020
524 288	795.068 ± 0.370	830.797 ± 0.121
1 048 576	3176.341 ± 3.635	3370.064 ± 4.832

La Figura 6.3 representa directamente la longitud de secuencia frente al tiempo medio de ejecución. El eje horizontal utiliza escala logarítmica en base 2 para separar uniformemente las longitudes ensayadas, que son potencias de dos, mientras que el eje vertical se mantiene **lineal** en milisegundos. De esta forma, la distancia vertical entre las curvas conserva la diferencia absoluta de tiempo y resulta más visible la separación entre CUDA y las rutas de referencia para secuencias grandes.

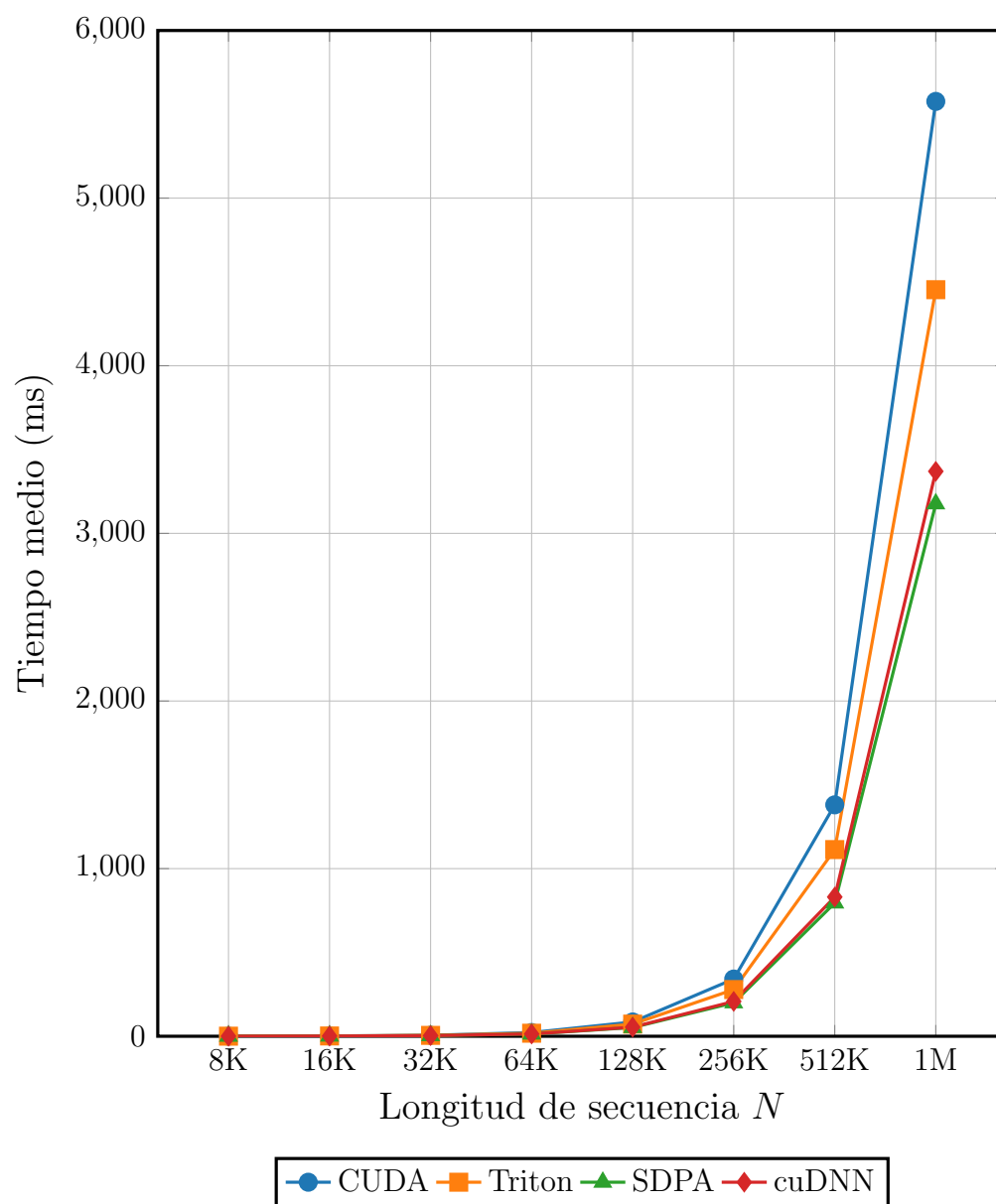


Figura 6.3: Tiempo medio de ejecución frente a longitud de secuencia, con eje vertical lineal.

Para $N = 8192$, SDPA es aproximadamente

$$\frac{0,579}{0,252} \simeq 2,30 \times \quad (6.3)$$

más rápido que CUDA. Para $N = 1\,048\,576$, la relación se reduce a

$$\frac{5576,997}{3176,341} \simeq 1,76 \times . \quad (6.4)$$

La reducción del factor al aumentar N es coherente con la hipótesis de que una parte importante de la penalización para $N = 8192$ procede de la falta de bloques suficientes para poblar la A100. Cuando N es muy grande, el *grid* deja de ser escaso y permanece una diferencia asociada a la eficiencia sostenida dentro del SM.

Triton estabiliza una ventaja menor sobre CUDA para tamaños grandes: aproximadamente $1,225 \times$, $1,240 \times$ y $1,252 \times$ para 256K, 512K y 1M, respectivamente. Además, en $N = 16384$ CUDA es ligeramente más rápida que Triton (1.633 ms frente a 1.702 ms), lo que confirma que no existe un único orden de rendimiento para todos los tamaños.

6.9. Escalabilidad y throughput sostenido

A partir de aproximadamente $N = 65536$, duplicar la longitud de secuencia multiplica el tiempo por un factor cercano a cuatro en las cuatro rutas. Este comportamiento concuerda con el coste cuadrático de la atención exacta:

$$T(N) \propto N^2. \quad (6.5)$$

Considerando aproximadamente $4N^2d$ operaciones de coma flotante para los productos QK^T y PV , con $d = 128$, se obtiene el throughput sostenido aproximado de la Tabla 6.10.

Cuadro 6.10: Throughput aproximado en el régimen de secuencias grandes.

Impl.	Throughput aproximado
CUDA	101 TFLOP/s
Triton	126 TFLOP/s
cuDNN	167 TFLOP/s
SDPA	177 TFLOP/s

La Tabla 6.10 permite separar dos efectos. Para $N = 8192$, CUDA sufre simultáneamente por la cantidad limitada de trabajo global y por su eficiencia

interna. En secuencias grandes existe trabajo suficiente para todos los SM y la diferencia restante se aproxima más a la eficiencia sostenida del kernel. El kernel propio se estabiliza alrededor de 100 TFLOP/s, mientras que Triton se sitúa en torno a 126 TFLOP/s y las rutas de biblioteca alcanzan valores mayores.

6.10. Lecciones aprendidas y posibles mejoras

Los perfiles muestran que una parte importante del trabajo de optimización de bajo nivel produjo el efecto buscado. En CUDA no aparecen sectores excesivos por falta de coalescencia, no hay conflictos de banco, no se observa *register spilling* y el consumo de registros y memoria compartida es inferior al de varias alternativas. Estas son mejoras reales y medibles.

La principal limitación se encuentra en un nivel superior de diseño. El kernel propio procesa 64 filas de Q por CTA y recorre K, V con un tile físico de 16 filas. Frente a Triton, cuyo *mapping* generado permite identificar un bloque de consultas mayor, esta configuración obtiene menor reutilización de K, V . En cuDNN no se reconstruye el *mapping* interno: el menor número de sectores solicitado es únicamente **consistente con una mayor reutilización efectiva**, no una demostración de un tamaño de tile concreto. Para el tamaño perfilado, CUDA también expone menos trabajo global que la ruta SDPA. En otras palabras, se optimizó intensamente una granularidad local que no era necesariamente la mejor granularidad global para una A100 con 108 SM.

Las líneas de mejora que se desprenden de los resultados son:

1. aumentar el número de filas de Q procesadas por CTA para reutilizar cada tile de K, V sobre más consultas;
2. estudiar una reorganización donde parte de Q se mantenga en memoria compartida si ello permite aumentar el tile sin disparar el consumo de registros;
3. agrupar varias iteraciones físicas de 16 filas de K, V antes de actualizar el *softmax online*, reduciendo la frecuencia de reescalados y sincronizaciones;
4. reducir la frecuencia o el alcance de las barreras entre *warps*;
5. explorar estrategias de partición que incrementen el paralelismo global en secuencias donde el número de CTAs resulta insuficiente.

La conclusión principal no es que una optimización concreta haya sido incorrecta, sino que el rendimiento final depende del equilibrio entre reutilización de datos, paralelismo, sincronización y capacidad para mantener alimentadas las unidades de ejecución. Un kernel puede presentar accesos coalescentes, cero conflictos de banco y bajo consumo de recursos y, aun así, quedar por detrás si la distribución global del trabajo no aprovecha adecuadamente el dispositivo.

Capítulo 7

Conclusiones

El desarrollo de este trabajo ha permitido estudiar FlashAttention desde dos niveles de abstracción muy diferentes. Por una parte, se ha implementado una versión en Triton, delegando una parte importante de las decisiones de bajo nivel al compilador. Por otra, se ha desarrollado directamente en CUDA un kernel específico para arquitecturas Ampere, controlando de forma explícita la distribución del trabajo entre *warps*, el movimiento de datos entre memoria global, memoria compartida y registros, las instrucciones `cp.async` y `ldmatrix`, el uso de Tensor Cores mediante `mma.sync` y la disposición de los datos en memoria compartida.

El desarrollo de la implementación CUDA ha requerido un esfuerzo considerablemente mayor. Durante este proceso se han identificado y corregido problemas relacionados con accesos globales no coalescentes, conflictos de banco, presión sobre el fichero de registros, disposición de los fragmentos utilizados por las instrucciones matriciales, sincronización entre *warps* y utilización de múltiples buffers en memoria compartida. Los perfiles finales muestran que muchas de estas optimizaciones alcanzaron el objetivo previsto: los accesos globales son coalescentes, no se producen conflictos de banco en la implementación CUDA, no existe *register spilling* y el consumo de registros y memoria compartida resulta incluso inferior al de algunas de las implementaciones de referencia.

Sin embargo, los resultados muestran también una de las principales lecciones obtenidas durante el trabajo: optimizar correctamente los detalles microarquitectónicos de un kernel no garantiza que la estrategia global de distribución del trabajo sea la adecuada.

La implementación CUDA procesa bloques relativamente pequeños de consultas y presenta una reutilización de K y V inferior a la observada directamente en el *mapping* generado por Triton. Al mismo tiempo, para determinados tamaños del problema genera una cantidad limitada de tra-

bajo independiente, impidiendo aprovechar completamente los recursos que teóricamente permitirían mantener varios CTAs residentes en cada SM. En cuDNN, el menor número de sectores solicitado es consistente con una mayor reutilización efectiva, pero el kernel interno no se reconstruye y por tanto no se atribuye esa diferencia a un tamaño de tile concreto. El perfil de SDPA muestra, por su parte, un número de bloques significativamente superior.

Durante el desarrollo preliminar se realizaron también pruebas informales sobre una RTX 3060 Laptop en las que el comportamiento relativo de la implementación CUDA parecía más favorable. Estas ejecuciones no se realizaron bajo el protocolo experimental final, no se documentaron con el mismo entorno, versiones, repeticiones y perfiles, y por tanto **no forman parte de la evaluación experimental ni se extrae de ellas ninguna conclusión**. Se conservan únicamente como una observación anecdótica que motivó la posterior evaluación controlada sobre la A100.

Las medidas realizadas para longitudes de secuencia crecientes permiten además matizar los resultados. A medida que aumenta N , la implementación CUDA dispone de un número mayor de CTAs y la diferencia respecto a las alternativas se reduce. Para las secuencias grandes ensayadas, el kernel desarrollado alcanza aproximadamente 100 TFLOP/s sostenidos, frente a aproximadamente 126 TFLOP/s de Triton, 167 TFLOP/s de cuDNN y 177 TFLOP/s de SDPA. Estas cifras permiten valorar el resultado de forma cuantitativa y sin recurrir a calificativos subjetivos.

En este sentido, el resultado del proyecto no debe medirse únicamente por cuál de las implementaciones obtiene el menor tiempo de ejecución. El desarrollo manual ha permitido comprender aspectos que permanecen ocultos al utilizar bibliotecas de alto nivel: cómo se distribuyen los fragmentos de una instrucción `mma.sync` entre los hilos de un *warp*, cómo se pueden utilizar transferencias asíncronas para construir un *pipeline*, cómo afecta el patrón de acceso a los bancos de memoria compartida, cómo interactúan registros, memoria compartida y ocupación, y por qué una mayor ocupación teórica no implica necesariamente un mayor rendimiento.

Asimismo, la comparación con Triton ha resultado especialmente útil. Triton permite expresar el mismo algoritmo con una cantidad de código considerablemente menor y alcanza aproximadamente 126 TFLOP/s en el régimen de secuencias grandes medido, lo que pone de manifiesto el valor de los compiladores especializados para programación GPU. Sin embargo, el desarrollo CUDA proporciona una visibilidad mucho mayor sobre las decisiones que finalmente determinan el rendimiento y permite interpretar con mayor profundidad el código generado por herramientas de más alto nivel.

7.1. Limitaciones

Los resultados de este TFG deben interpretarse dentro de un alcance experimental deliberadamente acotado. La evaluación principal se realiza sobre una única NVIDIA A100-SXM4-40GB y, por tanto, no demuestra que las mismas relaciones de rendimiento se mantengan en otras GPUs Ampere, en generaciones posteriores o en aceleradores de otros fabricantes. Además, la instancia se obtiene a través de Vast.ai y las frecuencias no se fijan manualmente; las desviaciones típicas observadas son pequeñas, pero no se dispone de un entorno de frecuencia completamente controlada.

La configuración estudiada utiliza BF16, dimensión de cabeza fija $d = 128$, $B = H = 1$, atención no causal y longitudes de secuencia múltiplos de 16. No se evalúan otras dimensiones de cabeza, varios batches o cabezas simultáneos, atención causal, dropout, secuencias arbitrarias ni formatos numéricos distintos. Tampoco se estudian backward, entrenamiento completo, inferencia autoregresiva con caché KV ni integración dentro de un modelo completo; las conclusiones se refieren exclusivamente al forward aislado analizado.

El perfilado microarquitectónico detallado con Nsight Compute corresponde a $N = 8192$. Las medidas temporales cubren un rango mucho mayor de longitudes, pero no debe asumirse que porcentajes concretos de ocupación, actividad Tensor Core o comportamiento de caché sean constantes para todos esos tamaños. Del mismo modo, el análisis de cuDNN se limita a las métricas observables del kernel generado; no se reconstruye su implementación interna y, por ello, las inferencias sobre reutilización se formulan como interpretaciones consistentes con los contadores, no como hechos sobre su *mapping*.

Finalmente, la validación funcional se basa en `torch.allclose` con una semilla fija y tolerancias de $2 \cdot 10^{-2}$. Esto es suficiente como comprobación de regresión para el desarrollo realizado, pero no constituye un estudio exhaustivo de estabilidad numérica. Una evaluación dedicada a precisión debería utilizar múltiples semillas, más familias de entradas y métricas de error máximo, medio y relativo.

7.2. Trabajo futuro

Como líneas de trabajo futuras, las principales modificaciones identificadas consisten en aumentar el tamaño del bloque de consultas procesado por CTA, estudiar el almacenamiento de Q en memoria compartida para incrementar la reutilización de K y V , reducir la frecuencia de las actualizaciones del *softmax online* y de las sincronizaciones, y explorar estrategias de partición que permitan generar mayor paralelismo global. Estas modificaciones impli-

carían cambios estructurales importantes sobre el kernel actual, pero atacan directamente las limitaciones observadas durante el análisis de los perfiles.

En definitiva, el trabajo confirma que FlashAttention es un problema especialmente adecuado para estudiar la programación de GPUs modernas, ya que obliga a considerar simultáneamente cómputo matricial, jerarquía de memoria, paralelismo, sincronización y utilización de recursos. Aunque el resultado final de la implementación CUDA no haya alcanzado el rendimiento esperado sobre la A100, el proceso de llegar hasta ese resultado ha permitido identificar con precisión por qué ocurre y qué decisiones deberían modificarse para continuar mejorándolo. Esa comprensión constituye, probablemente, uno de los resultados más importantes obtenidos durante el desarrollo del proyecto.

Anexos

Anexo A

Contexto complementario: mecanismo de atención y arquitectura GPT

Este anexo amplía el contexto matemático y conceptual de la operación estudiada en la memoria. El cuerpo principal se centra en la implementación y el rendimiento del kernel de atención no causal. Aquí se explica con más detalle qué expresa la fórmula de la atención, por qué resulta útil y cómo se integra en una arquitectura GPT. La máscara causal se presenta únicamente como una variante empleada por los modelos autoregresivos; no forma parte de la configuración principal de los benchmarks de este trabajo.

A.1. Qué quiere decir la fórmula de la atención

La atención parte de una idea sencilla: para actualizar la representación de una posición de una secuencia, el modelo calcula qué otras posiciones contienen información útil. Para una única cabeza y sin máscara, la atención escalada producto punto se expresa como

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V, \quad (\text{A.1})$$

donde Q , K y V representan las matrices de consultas, claves y valores, respectivamente, y d es la dimensión de la cabeza.

Antes de calcular la atención, la representación de cada token se proyecta a tres espacios distintos. Las consultas Q representan qué está buscando cada posición; las claves K representan qué tipo de información ofrece cada

posición; y los valores V contienen la información que se transmitirá si una posición recibe peso de atención. Esta separación permite que una posición busque información en un espacio distinto de aquel en el que la ofrece.

El producto QK^T compara cada consulta con todas las claves. Si se considera una consulta q_i y una clave k_j , su producto escalar puede escribirse como

$$q_i \cdot k_j = \|q_i\| \|k_j\| \cos \theta. \quad (\text{A.2})$$

Esta identidad muestra la relación con la similitud coseno: cuando dos vectores apuntan en direcciones parecidas, el coseno del ángulo es alto y el producto escalar tiende a ser mayor. Sin embargo, la atención no calcula únicamente similitud semántica en el espacio original de *embeddings*. Los tokens se transforman previamente mediante proyecciones entrenables, por lo que el producto escalar mide compatibilidad en espacios aprendidos de consulta y clave.

La división por $\sqrt{d}$ evita que las puntuaciones crezcan excesivamente al aumentar la dimensión de la cabeza. Sin este escalado, los valores de entrada al *softmax* podrían presentar magnitudes elevadas y producir distribuciones demasiado concentradas. El escalado estabiliza la normalización y facilita el entrenamiento.

El *softmax* convierte las puntuaciones en pesos positivos que suman uno en cada fila. Posteriormente, esos pesos multiplican a V . La salida de una posición es, por tanto, una combinación ponderada de los vectores de valor. Una posición con un peso alto influye de forma importante en la salida, mientras que una posición con un peso pequeño apenas contribuye.

En atención causal se introduce además una matriz de máscara M :

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} + M \right) V. \quad (\text{A.3})$$

Para un modelo autoregresivo, $M_{ij} = 0$ cuando $j \leq i$ y toma un valor equivalente a $-\infty$ cuando $j > i$. De esta forma, la posición i no puede asignar probabilidad a posiciones futuras. Esta restricción es propia del caso autoregresivo; la implementación evaluada en este TFG utiliza atención no causal para aislar el comportamiento del kernel denso y mantener una configuración común entre todas las rutas comparadas.

La formulación anterior también permite entender por qué la operación resulta costosa. Para una secuencia de longitud N , la matriz de puntuaciones contiene N^2 elementos por cabeza. Una implementación eficiente no debe limitarse a acelerar los productos matriciales: también debe reducir escrituras

intermedias, mantener estabilidad numérica y organizar los datos de forma que la GPU pueda sostener un elevado paralelismo.

A.2. Por qué la atención es importante

La atención es la parte del bloque *Transformer* que permite comunicación entre posiciones. Una red *feed-forward* aplicada de manera independiente a cada posición puede transformar cada token, pero no puede decidir por sí sola qué otros elementos de la secuencia resultan relevantes para la representación actual. La atención introduce precisamente esa comunicación mediante una mezcla adaptativa de información.

Esta capacidad es especialmente importante en lenguaje. Para interpretar o predecir un token, el modelo puede necesitar recuperar información que apareció mucho antes en la secuencia: una entidad mencionada previamente, una condición sintáctica, una referencia o una relación semántica. La atención proporciona un mecanismo directo para que esas dependencias influyan en la representación actual.

Desde el punto de vista computacional, la misma propiedad que hace útil a la atención introduce un patrón exigente. El número de comparaciones crece cuadráticamente con la longitud de secuencia y la operación combina productos matriciales, normalización y acumulación de valores. Por ello constituye un buen objeto de estudio para analizar jerarquía de memoria, paralelismo, formatos numéricos y utilización de unidades matriciales.

A.3. Encaje dentro de una arquitectura GPT

Una arquitectura GPT se compone de una pila de bloques *Transformer decoder*. El texto se tokeniza y cada identificador se transforma en un vector mediante una tabla de *embeddings*. A esta representación se incorpora información posicional para que el modelo pueda distinguir el orden de los tokens.

Cada bloque transforma después la secuencia mediante dos ramas principales. La primera es la atención causal multicabeza. Cada cabeza calcula relaciones sobre una subdimensión del estado interno y las salidas de todas las cabezas se combinan de nuevo en el espacio del modelo. La segunda rama es una red *feed-forward* aplicada de manera independiente a cada posición. Ambas ramas se conectan mediante normalización y conexiones residuales, lo que permite apilar un gran número de bloques manteniendo estabilidad durante el entrenamiento.

En GPT, la atención es causal porque el modelo se entrena para predecir el siguiente token y no puede utilizar información futura. La máscara cambia el dominio de posiciones permitidas, pero no modifica los componentes fundamentales estudiados en este TFG: productos QK^T y PV , normalización mediante *softmax*, movimiento de datos entre niveles de memoria y utilización de las unidades matriciales. Por esta razón, estudiar la variante no causal sigue proporcionando información directamente relevante sobre la implementación eficiente del núcleo de la operación.

Anexo B

Instalación, compilación y reproducción de los experimentos

Este anexo resume la organización del proyecto y los pasos necesarios para preparar el entorno, compilar las extensiones nativas, ejecutar las pruebas y reproducir el perfilado. El repositorio combina herramientas CMake/CUDA para la parte nativa y uv/Python para las implementaciones y pruebas de alto nivel.

B.1. Organización del proyecto

El flujo de trabajo se divide en los siguientes componentes:

- **CMake**: configura y compila el código C++/CUDA, las extensiones nativas y los tests de bajo nivel.
- **uv**: crea el entorno Python reproducible e instala las dependencias declaradas por el proyecto.
- **pytest**: ejecuta el test funcional, los benchmarks temporales y los tests preparados para *profiling*.
- **CTest**: ejecuta desde CMake los tests correspondientes a la parte C++/CUDA.
- **profile.sh**: automatiza las sesiones de NVIDIA Nsight Compute para CUDA, Triton, SDPA y SDPA-cuDNN.

- `environment_report.sh`: registra las características de hardware y las versiones de software relevantes para reproducir las mediciones.

Los artefactos de compilación se almacenan bajo el directorio `build/`. Los informes generados por Nsight Compute se almacenan en `profile/`, de forma que los resultados de perfilado quedan separados del código fuente y de los binarios compilados.

B.2. Requisitos

La configuración utilizada en la memoria emplea CUDA Toolkit 13.0, CMake, un compilador C++ compatible con C++20, Python gestionado mediante `uv`, PyTorch, Triton y NVIDIA Nsight Compute. La implementación CUDA final está especializada para la arquitectura Ampere y el preset suministrado configura como objetivo `sm_80`, correspondiente a la A100 utilizada en las mediciones.

El repositorio incluye un preset CMake denominado `angel-wsl`. En dicho preset se especifican, entre otros parámetros, la arquitectura CUDA objetivo y las rutas de los compiladores. Estas rutas dependen de la máquina concreta, por lo que el usuario puede modificar el preset para adaptarlo a su instalación. De la misma forma, si se utiliza otra GPU Ampere deberá revisarse la arquitectura indicada en `CUDA_ARCHITECTURES`.

B.3. Preparación del entorno Python

El entorno y las dependencias se sincronizan mediante:

```
uv sync
```

Una vez completado este paso, las herramientas Python del proyecto pueden ejecutarse mediante `uv run`, evitando depender de paquetes instalados globalmente en el sistema.

B.4. Configuración y compilación con CMake

La primera configuración del árbol de compilación puede realizarse con el preset incluido:

```
cmake --preset angel-wsl
```

La compilación se realiza mediante:

```
cmake --build --preset angel-wsl
```

Los ficheros generados permanecen dentro de `build/`. Si se modifica la ruta de CUDA, el compilador C++ o la arquitectura objetivo, debe actualizarse el preset antes de regenerar el proyecto.

B.5. Ejecución de tests

La batería Python completa puede prepararse y ejecutarse con:

```
uv sync && uv run pytest
```

Los tests de rendimiento utilizan la misma semilla pseudoaleatoria en todas las implementaciones. Concretamente, antes de generar Q , K y V se inicializan los generadores con semilla 0 mediante `torch.manual_seed(0)` y `torch.cuda.manual_seed_all(0)`. De esta manera, CUDA, Triton, SDPA y cuDNN se evalúan con entradas reproducibles generadas bajo el mismo criterio, evitando que diferencias en los datos aleatorios introduzcan una fuente adicional de variación entre benchmarks.

Los tests C++/CUDA integrados con CMake pueden ejecutarse mediante CTest desde el árbol de compilación:

```
ctest --test-dir build --output-on-failure
```

Si el preset utiliza un subdirectorio específico dentro de `build/`, el argumento de `--test-dir` debe apuntar al directorio generado por dicho preset.

B.6. Perfilado con Nsight Compute

El script `profile.sh` automatiza la captura de las regiones NVTX correspondientes a las cuatro rutas evaluadas. Para habilitar su ejecución y lanzar el perfilado se utiliza:

```
chmod u+x ./profile.sh && ./profile.sh
```

El script genera informes `.ncu-rep` en el directorio `profile/`. Estos informes pueden abrirse posteriormente con `ncu-ui` para analizar ocupación, registros, memoria compartida, tráfico de memoria, conflictos de banco, utilización de Tensor Cores y causas de espera de los *warps*.

Las duraciones obtenidas bajo Nsight Compute no se utilizan como benchmark temporal principal, ya que la herramienta puede instrumentar y repetir internamente el kernel. Las medidas de tiempo presentadas en la memoria proceden de los tests específicos basados en eventos CUDA.

B.7. Registro del entorno de ejecución

Para facilitar la reproducibilidad se incluye el script `environment_report.sh`. Su ejecución se realiza mediante:

```
chmod u+x ./environment_report.sh && ./environment_report.sh
```

El informe recoge información como sistema operativo, CPU visible, memoria del sistema, modelo de GPU, número de SM, capacidad de cómputo, frecuencias observadas, límite de potencia, driver NVIDIA, CUDA Toolkit, compiladores, CMake, Python, PyTorch, cuDNN, Triton y versión de Nsight Compute. Este procedimiento fue el utilizado para documentar el entorno de prueba descrito en la memoria.

B.8. Flujo recomendado para reproducir el trabajo

Un flujo completo de reproducción puede resumirse en los siguientes pasos:

1. Adaptar, si fuese necesario, el preset `angel-wsl` a las rutas de compilador y arquitectura CUDA de la máquina.
2. Ejecutar `uv sync` para crear el entorno Python.
3. Configurar y compilar mediante CMake.
4. Ejecutar `uv run pytest` y CTest para comprobar la correctitud antes de medir rendimiento.
5. Ejecutar `environment_report.sh` para registrar las condiciones del experimento.
6. Ejecutar los benchmarks temporales de `pytest` bajo las mismas condiciones y semilla.
7. Ejecutar `profile.sh` para generar los informes de Nsight Compute en `profile/`.

Esta separación entre compilación, validación, benchmark y perfilado evita mezclar los costes de instrumentación con las medidas de tiempo y facilita repetir el análisis en otra máquina modificando únicamente los parámetros dependientes del entorno.

Bibliografía

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser e I. Polosukhin, “Attention Is All You Need”, *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.03762.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever, “Language Models are Unsupervised Multitask Learners”, informe técnico de OpenAI, 2019. Documento en línea.
- [3] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”, *International Conference on Learning Representations (ICLR)*, 2024; preprint arXiv:2307.08691, 2023. Documento en línea.
- [4] M. Milakov y N. Gimelshein, “Online normalizer calculation for softmax”, CoRR, abs/1805.02867, 2018. Documento en línea.
- [5] NVIDIA, “NVIDIA Ampere Architecture In-Depth”, documentación técnica, 2020. Documento en línea. Consultado en septiembre de 2026.
- [6] NVIDIA, *CUDA C++ Programming Guide*, documentación oficial de CUDA. Documentación en línea. Consultado en septiembre de 2026.
- [7] NVIDIA, *Parallel Thread Execution ISA*, documentación oficial de PTX. Documentación en línea. Consultado en septiembre de 2026.
- [8] NVIDIA, *NVIDIA Blackwell Tuning Guide*, documentación oficial de CUDA. Documentación en línea. Consultado en septiembre de 2026.
- [9] AMD, “AMD CDNA Architecture”, documentación oficial de AMD Instinct. Documentación en línea. Consultado en septiembre de 2026.
- [10] Tenstorrent, “TT-Metalium: Advanced Topics – Tensix architecture”, documentación oficial. Documentación en línea. Consultado en septiembre de 2026.

- [11] M. Gschwind, H. P. Hofstee, B. Flachs, M. Hopkins, Y. Watanabe y T. Yamazaki, “Synergistic Processing in Cell’s Multicore Architecture”, *IEEE Micro*, 2006.
- [12] NVIDIA, *Nsight Compute Profiling Guide*, documentación oficial. Documentación en línea. Consultado en septiembre de 2026.
- [13] PyTorch Contributors, “Scaled Dot Product Attention y sdpa.kernel”, documentación oficial de PyTorch. Documentación SDPA. Selector de backends. Consultado en septiembre de 2026.
- [14] NVIDIA, “Scaled Dot Product Attention”, documentación de NVIDIA cuDNN Frontend. Documentación en línea. Consultado en septiembre de 2026.
- [15] Triton Contributors, “triton.autotune”, documentación oficial de Triton. Documentación en línea. Consultado en septiembre de 2026.
- [16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [17] T. Tran, “Writing Speed-of-Light Flash Attention for 5090 in CUDA C++”, blog técnico, 23 de agosto de 2025. Enlace al artículo. Consultado en agosto de 2026.
- [18] S. Li, “Flash Attention from Scratch”, serie técnica sobre FlashAttention 2 en NVIDIA Ampere, 2025. Enlace a la serie. Consultado en agosto de 2026.
- [19] NVIDIA, “CUTLASS: Efficient GEMM in CUDA”, documentación oficial de CUTLASS. Documentación en línea. Consultado en agosto de 2026.
- [20] A. Armbruster, “How To Write A Fast Matrix Multiplication From Scratch With Tensor Cores”, blog técnico, 10 de agosto de 2024. Enlace al artículo. Consultado en agosto de 2026.
- [21] MLC Community, “Modern GPU Programming For MLSys”, libro y material docente abierto sobre programación moderna de GPU para sistemas de aprendizaje automático, 2026. Sitio web del proyecto. Consultado en septiembre de 2026.
- [22] W.-m. W. Hwu, D. B. Kirk e I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4.^a ed., Morgan Kaufmann, 2022.